

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Procedurální generování dungeonů ve 2D hrách

Implementace a srovnání algoritmů v herním enginu Godot



2026

Vedoucí práce:
doc. RNDr. Jan Konečný, Ph.D.

Jesse Sadowý

Studijní program: Informační technologie,
kombinovaná forma

Bibliografické údaje

Autor: Jesse Sadowý

Název práce: Procedurální generování dungeonů ve 2D hrách (Implementace a srovnání algoritmů v herním engineu Godot)

Typ práce: bakalářská práce

Pracoviště: Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci

Rok obhajoby: 2026

Studijní program: Informační technologie, kombinovaná forma

Vedoucí práce: doc. RNDr. Jan Konečný, Ph.D.

Počet stran: 53

Přílohy: zdrojový kód aplikace

Jazyk práce: český

Bibliographic info

Author: Jesse Sadowý

Title: Procedural Generation of Dungeons in 2D Games (Implementation and Comparison of Algorithms in Godot Game Engine)

Thesis type: bachelor thesis

Department: Department of Computer Science, Faculty of Science, Palacký University Olomouc

Year of defense: 2026

Study program: Information Technologies, combined form

Supervisor: doc. RNDr. Jan Konečný, Ph.D.

Page count: 53

Supplements: application source code

Thesis language: Czech

Anotace

V herním enginu Godot práce implementuje a experimentálně porovnává pět algoritmů procedurálního generování dungeonových map. Porovnávanými metodami jsou Náhodné umístění místností a chodeb, BSP, Náhodná procházka, Buněčné automaty a generování Bludiště, hodnocené z hlediska času generování a pokrytí mapy průchozím prostorem. Výsledky ukazují výrazné rozdíly mezi metodami: BSP a Bludiště jsou nejrychlejší, Buněčné automaty dosahují nejvyššího pokrytí. Pro klasické dungeons jsou nejvhodnější BSP a Místnosti, pro jeskynní struktury Buněčné automaty a Náhodná procházka, pro labyrinty Bludiště.

Synopsis

Five algorithms for procedural dungeon map generation were implemented and experimentally compared in the Godot game engine: Random room placement, BSP, Drunkard's walk, Cellular automata, and Maze generation. Generation time and floor coverage were evaluated as the primary metrics. The results show significant differences between the methods: BSP and Maze proved to be the fastest, while cellular automata achieve the highest coverage. BSP and Rooms are best suited for classic dungeons, cellular automata and drunkard's walk for cave-like structures, and the Maze algorithm for labyrinthine layouts.

Klíčová slova: procedurální generování; dungeon; 2D hry; Godot; herní engine; algoritmus

Keywords: procedural generation; dungeon; 2D games; Godot; game engine; algorithm

Děkuji vedoucímu práce doc. RNDr. Janu Konečnému, Ph.D. za odborné vedení a cenné připomínky při vypracování této práce.

Odevzdáním tohoto textu jeho autor/ka místopřísežně prohlašuje, že celou práci včetně příloh vypracoval/a samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.

Obsah

1	Úvod	9
2	Současný stav řešené problematiky	10
2.1	Procedurální generování obsahu ve hrách	10
2.1.1	Vymezení důležitých pojmů	10
2.1.2	Výhody a nevýhody procedurálního generování	10
2.1.3	Přístupy k procedurálnímu generování	11
2.2	Historický vývoj procedurálního generování ve hrách	11
2.2.1	Beneath Apple Manor	12
2.2.2	Rogue	12
2.2.3	Elite	13
2.3	Generování dungeonů ve 2D hrách	14
2.3.1	Požadavky na generování dungeonových map	14
2.4	Přehled metod generování	14
2.4.1	Náhodné umístění místností a chodeb	15
2.4.2	Binary Space Partitioning	15
2.4.3	Metoda Náhodné procházky	16
2.4.4	Buněčné automaty	16
2.4.5	Metoda Bludiště	17
2.4.6	Další metody	18
2.5	Shrnutí způsobů generování	18
3	Experimentální část	20
3.1	Použité technologie	20
3.1.1	Herní engine Godot	20
3.2	Návrh systému generování	20
3.2.1	Reprezentace mapy	20
3.2.2	Struktura generátoru	21
3.2.3	Vizualizace mapy a hráče	22
3.2.4	Ovládání generování a zobrazení statistik	23
3.3	Implementace generovacích metod	24
3.3.1	Náhodné umístění místností a chodeb	24
3.3.2	Binary Space Partitioning	26
3.3.3	Metoda Náhodné procházky	27
3.3.4	Buněčné automaty	28
3.3.5	Metoda Bludiště	30
3.4	Metodika měření a vyhodnocení	31
4	Výsledky	33
4.1	Výsledky jednotlivých algoritmů	33
4.1.1	Místnosti a chodby	33
4.1.2	Binary Space Partitioning	34
4.1.3	Náhodná procházka	35

4.1.4	Buněčné automaty	36
4.1.5	Bludiště	37
4.2	Srovnání algoritmů	38
4.2.1	Průměrný čas generování	39
4.2.2	Průměrné pokrytí	40
4.2.3	Škálování s velikostí mapy	40
4.2.4	Kompromis mezi rychlostí a pokrytím	42
4.2.5	Rozptyl výsledků na velké mapě	42
4.3	Souhrnná tabulka výsledků	43
4.4	Shrnutí výsledků	44
5	Diskuse	45
5.1	Silné a slabé stránky jednotlivých metod	45
5.2	Zkušenosti z implementace	46
5.3	Souvislost s předchozím výzkumem	46
5.4	Možnosti dalšího rozšíření práce	47
	Závěr	48
	Conclusions	49
	A Odkaz na repozitář projektu	50
	B Zkratky	51
	C Obsah elektronických dat	52
	Literatura	53

Seznam obrázků

1	Schéma architektury systému generování dungeonových map . . .	22
2	Uživatelské rozhraní aplikace pro generování map	23
3	Ukázka mapy generované pomocí algoritmu náhodných místností a chodeb (45×45)	26
4	Ukázka mapy generované pomocí algoritmu BSP (45×45)	27
5	Ukázka mapy generované pomocí algoritmu Náhodné procházky (45×45)	28
6	Ukázka mapy generované pomocí algoritmu Buněčných automatů (45×45)	29
7	Ukázka mapy generované pomocí algoritmu Bludiště (45×45) . .	31
8	Místnosti a chodby – průměrný čas generování podle velikosti mapy	33
9	Místnosti a chodby – průměrné pokrytí podle velikosti mapy . . .	34
10	BSP – průměrný čas generování podle velikosti mapy	34
11	BSP – průměrné pokrytí podle velikosti mapy	35
12	Náhodná procházka – průměrný čas generování podle velikosti mapy	36
13	Náhodná procházka – průměrné pokrytí podle velikosti mapy . . .	36
14	Buněčné automaty – průměrný čas generování podle velikosti mapy	37
15	Buněčné automaty – průměrné pokrytí podle velikosti mapy . . .	37
16	Bludiště – průměrný čas generování podle velikosti mapy	38
17	Bludiště – průměrné pokrytí podle velikosti mapy	38
18	Srovnání průměrného času generování všech algoritmů	39
19	Srovnání průměrného času generování na mapě 150×150	39
20	Srovnání průměrného pokrytí všech algoritmů	40
21	Škálování času generování s počtem buněk mapy	41
22	Škálování pokrytí s počtem buněk mapy	41
23	Kompromis mezi rychlostí a pokrytím	42
24	Rozptyl času generování na mapě 150×150	43
25	Rozptyl pokrytí na mapě 150×150	43

Seznam tabulek

1	Srovnání vybraných metod generování dungeonových map	19
2	Ukázka reprezentace mapy v mřížce	21
3	Průměrný čas generování a pokrytí	44

1 Úvod

Generování dungeonových map je ustálenou problematikou procedurálního generování obsahu (PCG) v digitálních hrách. Dungeonové mapy musí splňovat požadavky na variabilitu a znovuhratelnost, ale také základní podmínky hratelnosti, jako jsou průchodnost, přiměřené rozložení prostoru a vhodné rozmístění herních prvků. Volba generovací metody přitom zásadně ovlivňuje výsledný vzhled mapy i herní zážitek hráče.

Oblast PCG ve hrách je teoreticky dobře zpracována v souhrnných pracích [1] a [2], které přístupy ke generování kategorizují a popisují jejich vlastnosti. Experimentální srovnání se dosud zaměřovala převážně na kombinace algoritmů pro umísťování místností a propojování chodbami [3].

Tato práce implementuje a experimentálně porovnává pět algoritmů generování dungeonových map v herním enginu Godot: Náhodné umístění místností a chodeb, BSP, Náhodnou procházku (Drunkard's Walk), Buněčné automaty a generování Bludiště. Práce algoritmy hodnotí z hlediska času generování a pokrytí mapy průchozím prostorem. Výsledky práce diskutuje z pohledu vhodnosti jednotlivých přístupů pro různé typy herního prostředí.

Práce je členěna do pěti kapitol: rešerše PCG a vybraných metod generování, návrh a implementace systému, naměřené výsledky, diskuse a závěr.

2 Současný stav řešené problematiky

2.1 Procedurální generování obsahu ve hrách

2.1.1 Vymezení důležitých pojmů

Procedurální generování ve hrách lze vymezit jako algoritmickou tvorbu herního obsahu s omezeným nebo nepřímým zásahem člověka. Herní obsah přitom zahrnuje široké spektrum prvků, například mapy, úrovně, pravidla, úkoly, zbraně či předměty. V závislosti na konkrétním návrhu může PCG ovlivňovat nejen prostorovou strukturu hry, ale také její mechaniky, obtížnost či samotný průběh hraní [1].

V současnosti se pojem generování obsahu spojuje s metodami umělé inteligence, tato práce se však zaměřuje výhradně na algoritmické postupy založené na předem definovaných pravidlech.

Pojem hra je v odborné literatuře obtížné jednoznačně vymezit, protože zahrnuje široké spektrum forem lišících se cíli, pravidly, mírou interaktivity i použitou platformou. Termín hra se v práci chápe jako digitální videohra běžící v reálném čase, ve které hráč interaguje s virtuálním prostředím přes definované herní mechaniky. Práce se zaměřuje především na 2D videohry určené pro osobní počítače, případně další běžné platformy.

2.1.2 Výhody a nevýhody procedurálního generování

Procedurální generování obsahu přináší řadu výhod. Mezi hlavní patří vysoká variabilita výsledného obsahu a podpora znovuhratelnosti, protože každé spuštění produkuje unikátní prostředí. Dalšími výhodami jsou snížení nároků na ruční tvorbu obsahu a možnost generování rozsáhlých herních světů s minimálními paměťovými nároky, jak demonstruje například hra *Elite* (viz podkapitola 2.2.3). Parametrizací generátoru lze navíc přizpůsobit výsledné prostředí různým herním situacím nebo obtížností [1, 2].

Využití PCG s sebou však nese také několik významných omezení. Jedním z hlavních problémů je nižší míra kontroly nad výsledným obsahem. U ručně navržených úrovní má vývojář plnou kontrolu nad tempem hry, rozmístěním výzev, předmětů i klíčových herních momentů. Naproti tomu u procedurálního generování je výsledná podoba světa závislá na implementovaném algoritmu a zvolených parametrech. To může vést k nevyváženým úrovním, nelogickému rozmístění prvků nebo situacím, které jsou příliš jednoduché či naopak extrémně náročné. Vývojář proto musí věnovat značné úsilí návrhu, ladění a testování generátoru, aby generátor dosahoval uspokojivé kvality výsledného obsahu.

Dalším rizikem je možnost vzniku repetitivního nebo uměle působícího obsahu. I přestože algoritmus generuje obsah automaticky a náhodně, vychází z omezené množiny předem definovaných pravidel. Pokud není systém navržen dostatečně variabilně, může hráč postupně rozpoznat opakující se vzorce, což může negativně ovlivnit herní zážitek. Tento problém je sice možné minimalizo-

vat vhodným návrhem algoritmu a důkladným laděním, nicméně zcela eliminovat jej bývá obtížné [1].

2.1.3 Přístupy k procedurálnímu generování

PCG lze klasifikovat podle více kritérií, například podle způsobu reakce na hráče nebo podle deterministické či stochastické povahy výstupu.

Nejčastěji se lze v praxi setkat s adaptivním či neadaptivním přístupem, a to vzhledem k chování hráče. Neadaptivní (statické) generování vytváří obsah bez ohledu na chování hráče a jeho průběh hry. Všechny generované struktury vycházejí ze stejných pravidel a reakce hráče na podněty neovlivní výsledek. Naopak adaptivní generování pracuje s informacemi o hráči. Systém může sledovat úspěšnost hráče, rychlost reakcí, délku průchodu úrovně, využívané herní prvky nebo způsob řešení situací a na základě těchto dat dále upravovat generovaný obsah.

Dalším důležitým rozdělením je deterministický a stochastický přístup. Deterministické generování vychází z pevně daných počátečních hodnot, například *seed*, což je počáteční hodnota generátoru pseudonáhodných čísel. Při stejném vstupu vždy produkuje stejný výstup. Tento přístup umožňuje reprodukovatelnost výsledků, což je výhodné například při testování nebo sdílení konkrétních map mezi hráči. Stochastické generování naproti tomu pracuje s náhodností bez pevně stanoveného počátečního stavu, takže výsledky se liší i při opakovaném spuštění algoritmu.

Z hlediska samotného procesu generování lze rozlišit konstruktivní metody a přístup označovaný jako *generate-and-test*. Konstruktivní metody vytvářejí obsah postupně podle předem definovaných pravidel a omezení. Každý krok generování přímo přispívá k vytvoření výsledné struktury. Naproti tomu metoda *generate-and-test* nejprve vytvoří potenciální řešení, které algoritmus následně ověří podle stanovených kritérií. Pokud výsledek nevyhovuje, proces se opakuje. Tento přístup může být výpočetně náročnější, ale umožňuje lépe kontrolovat výsledné vlastnosti mapy.

V literatuře se rovněž objevuje pojem *mixed-authorship* (smíšené autorství), kdy se na tvorbě obsahu podílí jak algoritmus, tak lidský návrhář. Systém může například generovat návrhy úrovně, které následně autor upravuje, případně může reagovat na částečně ručně vytvořenou strukturu. Tento přístup kombinuje kreativitu člověka s rychlostí automatizovaného generování.

Plně automatická generace naopak probíhá bez zásahu člověka během samotného vytváření obsahu. Tento způsob je typický zejména pro roguelike hry a další tituly, u nichž je cílem vysoká variabilita jednotlivých průchodů [1].

2.2 Historický vývoj procedurálního generování ve hrách

Počátky procedurálního generování sahají již do 70. let 20. století, kdy byl výpočetní výkon i dostupná operační paměť velmi omezené. Tyto limity výrazně ovlivnily množství ručně vytvářeného obsahu, a vývojáři proto hledali způsoby,

jak vytvářet variabilní herní prostředí s minimálními nároky na RAM. Právě z tohoto důvodu se začaly prosazovat procedurální přístupy [1].

2.2.1 Beneath Apple Manor

Jednou z prvních her, které implementovaly PCG, byla Beneath Apple Manor z roku 1978. Autorem titulu byl Don Worth a hru vydalo studio The Software Factory. Hra vznikala primárně pro počítač Apple II.

Beneath Apple Manor generovalo herní prostředí algoritmicky při každém novém spuštění. Díky tomu nebylo rozložení místností, chodeb ani jednotlivých herních prvků či nepřátel nikdy zcela totožné, a právě tím byla hra přívětivá pro znovuhratelnost. Cílem hry bylo získat zlaté jablko a postup bylo možné ukládat, takže po smrti hráče se načetla uložená úroveň znovu, avšak hráč přišel o část svých atributů.

Autor hry čerpal inspiraci z programu Dragon Maze, který rovněž využíval náhodné výpočty ke generování bludišť. Tento princip však dále rozvinul a navrhl algoritmus vytvářející funkční a hratelnou strukturu dungeonů složenou z místností propojených chodbami. Hra navíc umožňovala hráči ovlivnit některé parametry generování, například počet místností na jednotlivých patrech, barevnou variantu zobrazení nebo samotnou obtížnost. Po vytvoření struktury úrovně algoritmus náhodně rozmístil také nepřátele, poklady a další herní objekty. S rostoucí hloubkou dungeonů se zároveň zvyšovala obtížnost hry, a to zejména síla nepřátel.

Použitý algoritmický přístup lze zařadit mezi rané příklady způsobu generování založeného na místnostech a chodbách, který se později stal jedním ze základních modelů využívaných v hrách typu roguelike. Beneath Apple Manor tak představuje významný historický příklad využití těchto metod, jehož principy se dále rozvíjely v následujících dekáдах [4, 5].

2.2.2 Rogue

Hra Rogue si oproti výše zmíněnému Beneath Apple Manor získala výrazně větší popularitu a měla zásadní vliv na další vývoj podobných her. Právě podle této hry vznikl podžánr označovaný jako *roguelike*, z jehož principů dodnes vychází celá řada titulů.

Roguelike hry bývají typické náhodně generovanými úrovněmi, tahovým systémem, permanentní smrtí postavy (*permadeath*) a zpravidla i vyšší obtížností. Náhodné generování zde neplní pouze technickou funkci, ale představuje zásadní designový prvek, který podporuje průzkum, strategické rozhodování i adaptaci hráče na proměnlivé prostředí.

Hra vyšla v roce 1980 a jejími autory jsou Michael Toy, Glenn Wichman a Ken Arnold. Titul vznikl původně pro unixové systémy a zobrazoval herní svět pomocí ASCII znaků v textovém terminálu. Jednotlivé prvky prostředí znázorňovaly různé symboly, například hráč symbolem @, stěny znakem #, nepřátelé písmeny abecedy. Tento způsob zobrazení nebyl pouze důsledkem technických

omezení tehdejšího výpočetního výkonu, ale zároveň umožňoval poměrně flexibilní práci s herním světem a jeho generováním právě díky jednoduchosti.

Rogue výrazně zpopularizovalo princip permanentní smrti. Postup hrou nebylo možné ukládat a jakmile hráč zemřel, musel začít zcela od začátku. Hráči tak zůstávaly pouze nabyté zkušenosti a znalost herních mechanik. Smrt zde proto nepředstavovala pouze neúspěch, ale také nedílnou součást učení se hernímu systému.

Cílem celé hry je najít Amulet of Yendor, který se nachází v nejhlubším patře dungeonů, a následně se vrátit zpět na povrch. Dungeon obsahuje několik pater (tradičně 26) a každé z nich se generuje procedurálně. Patra se skládají z místností propojených chodbami a jejich rozložení se při každém novém průchodu mění. Hra náhodně rozmísťuje také nepřátele, pasti, poklady, zbraně a další předměty.

Prvek náhody se projevuje i ve vlastnostech předmětů. Například lektvary, svitky či prsteny mají při každé nové hře odlišné označení a jejich účinek není hráči předem znám. I pokud hráč nalezne stejný typ předmětu jako v předchozí hře, jeho vlastnosti se mohou lišit. To posiluje nejistotu a nutnost experimentování.

Kombinace procedurálně generovaných úrovní a permanentní smrti zajišťuje, že každý průchod hrou je unikátní. Hráč se tak nemůže spoléhat na zapamatování konkrétní mapy, ale musí se přizpůsobovat aktuálně vygenerovaným podmínkám. Právě tento princip se stal jedním ze základních stavebních kamenů celého roguelike žánru [6, 7].

2.2.3 Elite

Dalším významným titulem využívajícím procedurální generování je hra Elite z roku 1984, kterou vytvořili David Braben a Ian Bell. Na rozdíl od předchozích her se nejedná o dungeon, ale o vesmírný simulátor obchodování a soubojů. Přesto je Elite z hlediska procedurálního generování velmi důležitým milníkem.

Hra byla původně vydána pro počítač BBC Micro, který disponoval velmi omezenou pamětí. Z tohoto důvodu nebylo možné ukládat rozsáhlý herní svět přímo v paměti. Vývojáři proto zvolili přístup, kdy hra generovala herní svět algoritmicky na základě předem definovaného číselného základu (*seed*). Pomocí tohoto deterministického postupu bylo možné při každém spuštění znovu vytvořit stejnou galaxii bez nutnosti jejího explicitního uložení.

Elite obsahovalo osm galaxií a každá z nich zahrnovala stovky hvězdných systémů. Vlastnosti jednotlivých systémů, jako je jejich název, ekonomika, úroveň technologického rozvoje či typ vlády, generovaly pseudonáhodné algoritmy. Procedurálně vznikaly také ceny komodit a další herní parametry.

Tento přístup ukázal, že procedurální generování není omezeno pouze na tvorbu dungeonů nebo jednotlivých úrovní, ale může být využito i pro vytvoření rozsáhlých herních světů. Elite tak demonstrovalo, že algoritmické generování obsahu představuje efektivní řešení zejména v podmínkách s omezeným výpočetním výkonem a omezenými paměťovými zdroji [8].

2.3 Generování dungeonů ve 2D hrách

Dungeonové mapy se typicky skládají z místností a chodeb, obsahují jasně definované průchozí a neprůchozí oblasti a musí splňovat požadavky na hratelnost. Generování takových struktur proto vyžaduje algoritmy, které dokáží zajistit nejen variabilitu výsledku, ale i splnění základních strukturálních vlastností.

2.3.1 Požadavky na generování dungeonových map

Návrh algoritmu pro generování dungeonové mapy musí zohledňovat několik základních požadavků, které vycházejí jak z technických omezení, tak z hlediska hratelnosti. Generovaná mapa by neměla být pouze náhodným shlukem místností a chodeb, ale strukturovaným prostředím, které umožňuje smysluplný průchod.

Jedním ze základních požadavků je proto průchodnost mapy neboli *connectivity*. Všechny klíčové části mapy by měly být vzájemně dosažitelné, aby nevznikaly izolované oblasti bez přístupu. Tento požadavek je zásadní zejména u her, kde je cílem projít celou úroveň nebo nalézt konkrétní objekt. Výjimkou jsou hry, ve kterých je neprůchodnost záměrná – hráč se do izolovaných částí dostane například prokopáním.

Dalším důležitým aspektem je kontrola struktury a rozložení prostoru. Mapa by měla obsahovat přiměřený poměr otevřených a uzavřených prostor, vhodnou velikost místností a dostatečně široké průchody, aby hráč mohl bez problémů projít. Příliš husté nebo naopak příliš prázdné prostředí může negativně ovlivnit hratelnost i orientaci hráče.

Významným požadavkem je rovněž variabilita výsledků. Opakované spuštění generátoru by mělo vést k dostatečně odlišným mapám, aby generování zachovalo prvek znovuhratelnosti. Zároveň je však vhodné, aby generování respektovalo určitá pravidla nebo omezení, která zajistí konzistentní kvalitu výsledku.

Z technického hlediska je důležitá také výpočetní náročnost algoritmu. Generování by mělo probíhat v přiměřeném čase a nemělo by výrazně zatěžovat systém, zejména pokud probíhá za běhu hry (online generování). U jednodušších 2D dungeonových her je obvykle možné využít algoritmy s relativně nízkou časovou složitostí, jelikož požadavky na generování jsou značně nižší.

Možnost nastavit velikost mapy, počet místností, hustotu chodeb nebo další vlastnosti generátoru umožňuje lépe přizpůsobit výsledné prostředí konkrétním požadavkům hry a usnadňuje testování různých konfigurací [1, 2].

2.4 Přehled metod generování

Metody generování dungeonových map se liší způsobem konstrukce prostoru, mírou kontroly nad výslednou strukturou, typem generovaného prostředí a výpočetní náročností. Některé algoritmy vytvářejí mapu postupně přidáváním jednotlivých prvků, jiné pracují s rozdělováním prostoru nebo s pravidly aplikovanými na mřížku buněk.

Ve 2D hrách převažují metody založené na náhodném umístění místností a jejich následném propojování, rozdělování velkého prostoru na menší části nebo algoritmy založené na takzvané Náhodné procházce. Každý z těchto přístupů má své výhody i omezení a je vhodný pro odlišný typ herního prostředí.

2.4.1 Náhodné umístění místností a chodeb

Generování pomocí náhodného umístění místností a chodeb patří mezi nejpoužívanější a zároveň nejjednodušší přístupy z hlediska implementace.

V mapě nejprve vznikají místnosti různých velikostí, obvykle obdélníkového nebo čtvercového tvaru. Během generování se kontroluje, aby se nově vznikající místnost nepřekrývala s již existujícími. Výsledkem je množina místností reprezentovaných například souřadnicemi a rozměry. Místnosti se následně propojí chodbami. Ty tvoří jednoduché horizontální a vertikální průchody spojující středy dvou místností nebo jejich nejbližší body. Častým řešením je také propojení ve tvaru písmene L, kdy dvě místnosti spojuje dvojice kolmých chodeb.

Pro zajištění průchodnosti mapy a dosažitelnosti všech místností se často využívají algoritmy z oblasti teorie grafů. Jedním z typických řešení je použití minimální kostry grafu (Minimum Spanning Tree), která zaručuje propojení všech místností při minimálním počtu chodeb.

Výhodou této metody je především její jednoduchost. Generované dungeony obvykle obsahují přehledně oddělené místnosti a srozumitelnou síť propojení. Nevýhodou může být menší variabilita, protože místnosti mají často pravidelný tvar a prostředí tak může působit příliš uniformně. Dalším problémem bývá méně efektivní využití prostoru, kdy mezi jednotlivými místnostmi zůstávají větší nevyužité oblasti. Komplikace může přinášet i samotné propojování. U jednodušších implementací mohou vznikat kolize chodeb, nevhodné křížení nebo příliš dlouhé průchody.

Z těchto důvodů vývojáři metodu často kombinují s dalšími postupy nebo doplňují o dodatečná pravidla, která omezují velikosti a vzájemné vzdálenosti místností, délku chodeb nebo celkové rozložení prostoru [1, 2].

2.4.2 Binary Space Partitioning

Metoda Binary Space Partitioning (BSP) patří mezi přístupy založené na systematickém rozdělování prostoru. Celá mapa se nejprve reprezentuje jako jeden velký obdélníkový nebo čtvercový prostor, tedy kořen stromu, který je postupně rozdělován na menší části. Každé rozdělení může probíhat horizontálně nebo vertikálně a výsledkem je hierarchická struktura podobná binárnímu stromu.

Generování obvykle začíná rozdělením celé mapy na dvě části. Tyto části se dále rekurzivně dělí, dokud nedosáhnou určité minimální velikosti. Každé rozdělení vytváří dva nové uzly ve stromové struktuře; listové uzly představují konečné oblasti. Takto vzniklá stromová reprezentace umožňuje přehledně uchovávat strukturu rozdělení prostoru.

V každé vzniklé oblasti vznikne místnost o trochu menších rozměrech, než je daná oblast. Tím vzniká přirozený prostor mezi místnostmi, který lze využít například pro generování chodeb nebo dalších herních prvků. Místnosti se propojují postupným spojováním sousedních uzlů stromové struktury, podobně jako u metody náhodného umístění místností a chodeb.

Díky tomuto postupu lze relativně snadno zajistit vzájemnou dosažitelnost všech místností. Zásadní výhodou BSP je dobrá kontrola nad velikostí a rozmístěním jednotlivých částí mapy. Parametry dělení prostoru umožňují ovlivnit výsledný vzhled prostředí a předejít situacím, kdy by vznikaly příliš malé nebo naopak příliš velké místnosti.

Nevýhodou této metody může být opět poměrně pravidelný a místy uniformní vzhled generovaného prostředí, protože výsledná struktura přímo odráží způsob dělení prostoru [1, 2].

2.4.3 Metoda Náhodné procházky

Metoda Náhodné procházky, často označovaná také jako Drunkard's Walk, je založena na principu postupného rozšiřování prostoru pomocí pohybu náhodným směrem. Algoritmus začíná na předem dané počáteční pozici na mapě a v každém kroku se náhodně přesune do jednoho ze sousedních polí. Směr pohybu je vybírán ze čtyř základních směrů v mřížce, tedy nahoru, dolů, doleva nebo doprava.

Při každém kroku algoritmus označí aktuální buňku jako průchozí prostor. Postupným opakováním tohoto procesu vzniká síť chodeb a otevřených oblastí, jejichž tvar závisí na délce a průběhu Náhodné procházky. Výsledná struktura mapy bývá velmi nepravidelná a rozdílná při každém generování.

Generování obvykle pokračuje po předem stanovený počet kroků nebo dokud průchozí buňky nedosáhnou stanoveného poměru plochy. V některých variantách algoritmu může být použito více nezávislých „chodců“, kteří se po mapě pohybují současně a vytvářejí tak komplexnější strukturu prostředí.

Výhodou této metody je velmi jednoduchá implementace a schopnost generovat přirozeně působící struktury. Nevýhodou je menší kontrola nad výsledným tvarem mapy a obtížnější zajištění rovnoměrného využití prostoru. V některých případech průchozí oblasti vznikají v blízkosti počáteční pozice algoritmu, zatímco jiné části mapy zůstávají téměř nevyužité.

Literatura metodu označuje jako vhodnou pro generování organicky působících prostředí připomínajících jeskyně [1].

2.4.4 Buněčné automaty

Buněčné automaty představují přístup k procedurálnímu generování založený na aplikaci specifických pravidel na mřížku buněk. Tento přístup reprezentuje mapu jako dvourozměrnou mřížku, kde každá buňka může být například ve stavu podlaha nebo prázdný prostor. Na začátku generování algoritmus inicializuje mapu náhodným rozdělením těchto stavů, což vytváří základní strukturu pro další postup.

Následně se celá mapa prochází a stav jednotlivých buněk se aktualizuje na základě stavu jejich sousedních buněk. Nejčastěji algoritmy používají takzvané Mooreovo sousedství, které zahrnuje osm buněk obklopujících aktuální buňku. Pravidla generování bývají různá. Mohou například určovat, že buňka zůstane průchozí pouze tehdy, pokud má dostatečný počet průchozích sousedů. Buňka zůstane prázdnou, nebo se změní na stěnu, pokud ji obklopuje větší množství prázdného prostoru.

Opakováním aplikováním těchto pravidel se náhodně vytvořená struktura postupně vyhlazuje. Izolované buňky nebo malé skupiny buněk zmizí a vznikají větší souvislé oblasti. Výsledkem jsou mapy s nepravidelnými tvary, které často připomínají přirozené jeskynní systémy.

Výhodou této metody je schopnost generovat organicky působící prostředí s nepravidelnými tvary, které se liší od klasických dungeonů. Algoritmus je výpočetně efektivní: každý simulační krok prochází mapu v lineárním čase vzhledem k počtu buněk. Nevýhodou může být menší kontrola nad přesnou strukturou mapy a také nutnost dodatečných kroků, které zajistí průchodnost celé mapy. Implementace může generovat izolované části a v některých případech je proto nutné je zcela odstranit nebo dodatečně propojit, což negativně ovlivňuje složitost algoritmu [1, 2].

2.4.5 Metoda Bludiště

Metody generování bludišť vytvářejí struktury tvořené převážně úzkými chodbami, které tvoří souvislou síť průchodů. Algoritmus reprezentuje mapu jako mřížku buněk oddělených stěnami. Samotné generování spočívá v postupném odstraňování těchto stěn tak, aby vznikly propojené průchody.

Tyto algoritmy často vycházejí z principů grafových algoritmů, například prohledávání do hloubky (Depth-First Search) s návratem (Backtracking) nebo algoritmů typu Prim či Kruskal. V případě algoritmu založeného na prohledávání do hloubky se začíná v náhodně zvolené buňce, ze které se postupně prochází do sousedních dosud nenavštívených buněk. Při každém kroku se odstraní stěna mezi aktuální buňkou a vybraným sousedem. Pokud již neexistuje žádný nenavštívený soused, algoritmus se vrací zpět k předchozí buňce.

Primův algoritmus vytváří bludiště postupným rozšiřováním již vytvořené oblasti. Algoritmus začíná v náhodné buňce a postupně vybírá sousední stěny, které oddělují navštívené buňky od nenavštívených. Pokud odstranění vybrané stěny spojí dvě dosud nepropojené oblasti, algoritmus tuto stěnu odstraňuje a nová buňka se stane součástí bludiště.

Kruskalův algoritmus naproti tomu pracuje s množinou všech stěn v mapě. Stěny jsou postupně vybírány v náhodném pořadí a odstraňují se pouze tehdy, pokud jejich odstranění nespojí dvě buňky, které již patří do stejné části bludiště. Tím se postupně vytváří souvislá síť průchodů.

Výsledkem je perfektní bludiště, ve kterém mezi dvěma body existuje právě jedna cesta. Taková struktura neobsahuje cykly a vytváří spleť chodeb.

Tento typ generování je vhodný zejména pro hry zaměřené na průzkum a orientaci v prostoru.

Nevýhodou této metody je omezený výskyt větších otevřených prostor, protože generovanou strukturu tvoří převážně úzké chodby. Z tohoto důvodu ji dungeonové hry často kombinují s jinými metodami, které umožňují vytvářet také větší místnosti. V některých případech se proto po vygenerování bludiště odstraní vybrané stěny, čímž vznikají cykly a otevřenější struktura mapy [1, 2].

2.4.6 Další metody

Kromě výše uvedených přístupů existuje celá řada dalších metod generování dungeonových map. Patří mezi ně například generování založené na grafech, kde algoritmus strukturu mapy nejprve reprezentuje jako graf. Jednotlivé vrcholy grafu představují místnosti a hrany určují jejich vzájemné propojení. Na základě takto vytvořené abstraktní struktury algoritmus vytvoří konkrétní geometrickou podobu mapy.

Další možností je využití procedurálních gramatik, které definují pravidla pro vytváření jednotlivých částí mapy. Pravidla se aplikují postupně, podobně jako při tvorbě vět ve formálních jazycích. Tento přístup umožňuje poměrně přesně řídit strukturu generovaného prostředí a je vhodný zejména pro hry, které vyžadují specifické uspořádání jednotlivých herních prvků.

V literatuře se rovněž objevují přístupy založené na evolučních algoritmech nebo optimalizačních metodách, které se snaží generované mapy postupně vylepšovat podle specifických kritérií. Zmíněné metody pracují s populací kandidátních map, které algoritmus postupně upravuje pomocí genetických operací (mutace, křížení). Algoritmus výsledné mapy hodnotí podle zvolených metrik, například průchodnosti, velikosti prostoru nebo rozložení herních prvků.

Modernější přístupy zahrnují také metody založené na splňování různých omezení. Jedná se o takzvanou constraint-based generation nebo algoritmy typu Wave Function Collapse, které generují mapy postupným výběrem kompatibilních prvků z předem definované množiny vzorů. Tyto metody umožňují generovat velmi rozmanité struktury, avšak jejich implementace bývá značně složitější.

Uvedené přístupy jsou často výpočetně náročnější než základní algoritmy popsané v předchozích podkapitolách, a proto se v praxi využívají spíše při offline generování obsahu nebo při návrhu komplexnějších herních světů [1, 2].

2.5 Shrnutí způsobů generování

Jednotlivé přístupy se liší především způsobem konstrukce prostoru, mírou kontroly nad výslednou strukturou a typem generovaného prostředí. Zatímco některé metody vytvářejí spíše pravidelné struktury místností a chodeb, jiné generují nepravidelná a organicky působící prostředí.

Základní charakteristiky a využití popsaných metod shrnuje tabulka 1.

Metoda	Kontrola struktury	Variabilita	Typ prostředí
Místnosti a chodby	vysoká	střední	klasické dungeons
BSP	vysoká	střední	strukturované mapy
Buněčné automaty	nízká	vysoká	jeskyně
Náhodná procházka	nízká	vysoká	organické struktury
Bludiště	střední	střední	labyrinty

Tabulka 1: Srovnání vybraných metod generování dungeonových map

3 Experimentální část

Práce implementuje pět algoritmů z řešební části: generování pomocí Místností a chodeb, Binary Space Partitioning, Buněčné automaty, Náhodnou procházku a generování Bludiště.

3.1 Použité technologie

3.1.1 Herní engine Godot

Pro vizualizaci generovaných map práce využívá herní engine Godot. Jedná se o volně dostupný (open-source) engine určený pro vývoj 2D i 3D her. Poskytuje integrované nástroje pro práci s grafikou, fyzikou, scénami i skriptováním, což umožňuje rychlé vytváření a testování herních prototypů.

Významnou vlastností Godotu je podpora dlaždicových map (TileMap), které jsou při vývoji 2D her často využívány a výrazně usnadňují samotné grafické ztvárnění. Tento způsob reprezentace je zároveň vhodný i pro implementaci algoritmů procedurálního generování, protože poskytuje jednoduchou manipulaci s jednotlivými buňkami mapy. Engine zároveň umožňuje snadné spouštění a testování ztvárněných metod.

Implementace algoritmů využívá integrovaný skriptovací jazyk GDScript, který je syntakticky podobný jazyku Python. GDScript zajišťuje přímý přístup k objektům herní scény, komponentám enginu i datovým strukturám, což usnadňuje rychlé prototypování generovacích metod a jejich průběžné ladění přímo v prostředí [9, 10].

3.2 Návrh systému generování

3.2.1 Reprezentace mapy

Všechny implementované metody generování pracují nad společnou datovou reprezentací mapy ve formě dvourozměrného pole. Mapa je uložena jako pole řádků, kde každý prvek odpovídá jedné buňce dungeonové mapy. Jednotlivé buňky obsahují celočíselnou hodnotu určující typ dlaždice.

Implementace rozlišuje tři základní stavy buněk: prázdný prostor (0), podlahu (1) a stěnu (2). Tyto hodnoty slouží jako základ pro vykreslení výsledné mapy.

Hodnota 0 se během generování používá především jako výchozí stav mapy, zatímco hodnoty 1 a 2 představují výslednou strukturu prostředí.

Tabulka 2 ukazuje příklad jednoduché reprezentace mapy: průchozí oblast (1) obklopenou stěnami (2), zatímco zbytek mapy tvoří nevyužitý prostor reprezentovaný hodnotou 0. Mapa má velikost 5×5 .

Velikost mapy určují dva parametry, šířka a výška, které se předávají generovacím algoritmům při jejich spuštění. Jednotlivé metody následně vytvářejí a upravují strukturu mapy podle svého principu generování.

0	0	0	0	0
0	2	2	2	0
2	2	1	2	2
2	1	1	1	2
2	2	2	2	2

Tabulka 2: Ukázka reprezentace mapy v mřížce

Zvolený způsob reprezentace sjednocuje všechny implementované algoritmy, protože každá metoda vrací výslednou mapu ve stejném formátu bez ohledu na svůj vnitřní princip.

3.2.2 Struktura generátoru

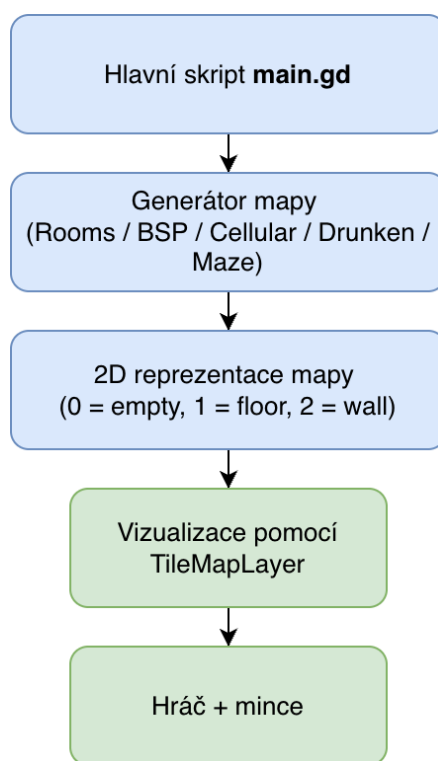
Systém vznikl jako modulární celek, který umožňuje jednotlivé algoritmy snadno rozšiřovat a vzájemně porovnávat. Každá metoda je tedy implementována v samostatném skriptu a využívá funkci `generate(width, height)`, která pro zadané rozměry vrací datovou reprezentaci mapy s výše zmíněnými hodnotami.

Díky tomuto sjednocení lze do aplikace přidávat další generátory bez zásahů do ostatních částí systému. Vytvořený prototyp tak zahrnuje implementaci všech zvolených metod.

Hlavní řídicí skript nese název `main.gd` a zajišťuje propojení uživatelského rozhraní s generovacími algoritmy. Skript na začátku definuje konstanty pro typy dlaždic (`EMPTY`, `FLOOR`, `WALL`), výchozí rozměry mapy (60×60) a pozice odpovídajících grafických prvků v atlasu dlaždic. Při spuštění scény (funkce `_ready()`) skript naplní rozbalovací nabídku názvy dostupných algoritmů a nastaví výchozí rozměry mapy.

Skript funguje jako prostředník mezi uživatelským rozhraním a jednotlivými generátory. Na základě volby uživatele zavolá příslušný generátor, převezme výslednou mapu a předá ji vizualizační vrstvě. Konkrétní průběh generování a měření popisuje podkapitola věnovaná ovládání aplikace.

Obrázek 1 schematicky znázorňuje architekturu navrženého systému.



Obrázek 1: Schéma architektury systému generování dungeonových map

3.2.3 Vizualizace mapy a hráče

Pro vizualizaci generovaných map aplikace využívá grafické prvky, takzvané *assets*. Tyto *assets* slouží především k přehlednému zobrazení struktury generované mapy a usnadňují orientaci uživatele. Vizualizace zahrnuje grafiku podlahy, stěn, postavy hráče a sbíratelných objektů.

Mapu zobrazuje komponenta `TileMapLayer`. Po dokončení generování skript předá datovou reprezentaci mapy funkci

`_draw_map_animated()`, která ji postupně převádí na odpovídající dlaždice, řádek po řádku s krátkou prodlevou mezi jednotlivými řádky. Aplikace mapuje jednotlivé typy buněk na konkrétní grafické prvky představující podlahu nebo stěnu. Tento způsob postupného vykreslování sice není nezbytný z hlediska samotného generování, ale výsledná vizualizace působí přívětivěji.

Součástí prototypu je také jednoduchá reprezentace hráče, která slouží především k ověření průchodnosti mapy. Umístování hráče zajišťuje funkce `_place_player_on_floor()`, která nejprve pomocí prohledávání do šířky (BFS) nalezne největší souvislou oblast podlahových dlaždic. V této oblasti poté hledá pozici s volným okolím 3×3 dlaždic, aby hráč nebyl ihned po vytvoření mapy zaseknut ve stěně. Pokud taková pozice neexistuje, funkce jako záložní řešení vybere náhodnou dlaždici z největší oblasti. S hráčem se lze jednoduše pohybovat pomocí kláves W, A, S, D. Po mapě se uživatel může pohybovat pravým tlačítkem myši, aniž by ovlivnil pozici hráče, pro návrat k původní pozici slouží

klávesa R.

Pro doplnění testovacího prostředí implementace přidává do mapy sbíratelné objekty ve formě mincí. Funkce `_place_coins_on_floor()` náhodně rozmístí 5 až 50 mincí na podlahové dlaždice v největší souvislé oblasti mapy. Pokud hráč posbírá všechny mince, automaticky se vygeneruje nová mapa se stejnými parametry, jako v případě předchozího generování. Tyto prvky neslouží jako hlavní součást PCG, jedná se pouze o grafické doplnění.

Implementace využívá volně dostupný balíček 2D Pixel Dungeon Asset Pack, jehož autorem je grafik vystupující pod jménem Pixel_Poem. Tento balíček obsahuje základní sadu grafických prvků určených právě pro tvorbu dungeonových 2D her [11].

Použité assety mají podobu pixelové grafiky v top-down perspektivě, tedy seshora dolů, a svým návrhem přímo odpovídají práci s dlaždicovou mapou, kterou engine využívá. Balíček je volně dostupný a lze jej využít v nekomerčních i komerčních projektech.

Grafická vrstva – hráč a mince – slouží pouze jako podpůrný nástroj pro vizualizaci výsledků generování.

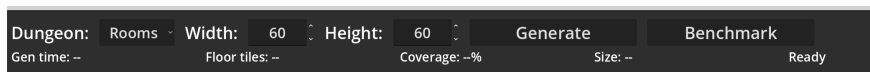
3.2.4 Ovládání generování a zobrazení statistik

Uživatelské rozhraní umožňuje výběr konkrétní generovací metody a nastavení základních parametrů mapy, šířky a výšky.

Rozhraní obsahuje dvě hlavní tlačítka, *Generate* a *Benchmark*. Po stisknutí tlačítka *Generate* skript na základě uživatelské volby vybere příslušný generátor pomocí příkazu `match` a zavolá jeho funkci `generate(width, height)`. Dobu generování měří funkce `Time.get_ticks_usec()` výhradně při volání generovací funkce, nikoli při vizualizaci mapy nebo umísťování herních objektů, čímž vykreslování neovlivní naměřený čas. Ihned po dokončení generování skript spočítá počet podlahových dlaždic a procentuální pokrytí mapy a zobrazí tyto údaje spolu s časem generování v uživatelském rozhraní. Výsledná mapa je poté vizualizována postupným vykreslováním jednotlivých řádků.

Tlačítko *Benchmark* automaticky testuje všechny implementované metody. Funkce `run_benchmark()` postupně spustí všech pět algoritmů pro každou z pěti definovaných velikostí map a každou konfiguraci opakuje desetkrát. V tomto režimu se mapa nevizualizuje a herní objekty se neumísťují.

Funkce `_log_results()` po každém běhu uloží výsledky (algoritmus, rozměry, čas generování, pokrytí, počet podlahových dlaždic) do souboru `results.csv`; její strukturu podrobněji popisuje podkapitola věnovaná metodice měření.



Obrázek 2: Uživatelské rozhraní aplikace pro generování map

3.3 Implementace generovacích metod

Každý algoritmus z kapitoly 2.4 je implementován jako samostatný skript v adresáři `res://dungeons/algorithms`.

Všechny generátory sdílejí několik společných implementačních prvků. Každý skript rozšiřuje třídu `RefCounted`, nemá tedy vazbu na scénu, a definuje trojici celočíselných konstant pro typy dlaždic: `TILE_EMPTY = 0` (prázdný prostor), `TILE_FLOOR = 1` (podlaha) a `TILE_WALL = 2` (stěna). Vstupním bodem je vždy statická funkce `generate(width, height)`, která vrací dvourozměrné pole představující výslednou mapu.

Na začátku funkce `generate()` každý generátor vytvoří lokální instanci `RandomNumberGenerator` a zavolá metodu `rng.randomize()`, čímž zajistí unikátní průběh náhodných operací bez ovlivnění zbytku aplikace. Následně inicializuje prázdnou mapu o rozměrech `width × height`, kde má každá buňka hodnotu `TILE_EMPTY`. Výjimkou je generátor Buněčných automatů, který inicializaci mapy a náhodnou počáteční výplň provádí v jednom kroku. Podrobněji tento postup popisuje příslušná podkapitola.

Závěrečným krokem každý generátor doplní stěny funkcí `_add_walls()`, jejíž implementace je ve všech skriptech totožná. Funkce prochází celou mapu a pro každou dlaždici podlahy kontroluje osm okolních buněk (Mooreovo sousedství). Pokud se v sousedství nachází buňka `TILE_EMPTY`, je změněna na `TILE_WALL`. Tímto postupem vznikne souvislé ohraničení všech průchozích částí mapy bez nutnosti generovat stěny explicitně během samotného algoritmu. Kontrola hranic mapy zajišťuje, že se při zápisu stěn nepřistupuje mimo rozsah pole.

3.3.1 Náhodné umístění místností a chodeb

Implementace algoritmu se nachází v souboru `rooms_generator.gd`.

Počet generovaných místností vychází z celkové plochy mapy podle vzorce `room_count = max(5, width * height / AREA_PER_ROOM)`. Hodnota konstanty `AREA_PER_ROOM = 360` vychází z odhadu průměrné plochy, kterou jedna místnost včetně svého okolí v mapě zabírá. Algoritmus generuje šířku i výšku místnosti náhodně v rozmezí 6 až 16 dlaždic, průměrná velikost místnosti tedy činí přibližně $11 \times 11 = 121$ dlaždic.

Kromě samotné plochy místnosti je však nutné počítat také s prostorem, který zabírají chodby, stěny a povinné jednopolíčkové mezery mezi místnostmi. Na základě empirického pozorování je tento faktor přibližně trojnásobkem plochy místnosti, tedy $121 \times 3 \approx 360$ dlaždic na jednu místnost. Na malých mapách tak algoritmus vygeneruje alespoň pět místností, zatímco na velkých mapách počet místností úměrně roste s plochou. Maximální počet pokusů o umístění místnosti odpovídá hodnotě `room_count * 10`, což zajišťuje dostatečný prostor pro opakované pokusy v případě kolizí s již umístěnými místnostmi, a zároveň zabraňuje nekonečnému cyklování na mapách, kde se požadovaný počet místností nedaří umístit.

Algoritmus reprezentuje každou novou místnost obdélníkem typu `Rect2`

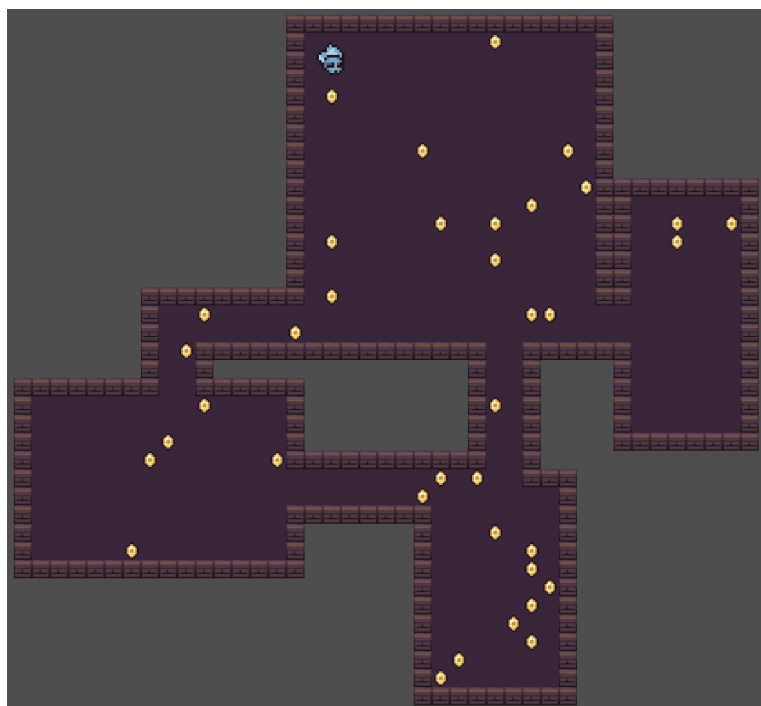
s náhodně vygenerovanými rozměry a určí pozici levého horního rohu tak, aby místnost ležela uvnitř hranic mapy a nezačínala přímo na jejím okraji.

Před přidáním nové místnosti do mapy algoritmus zkontroluje překryv se všemi dříve umístěnými místnostmi. Tato kontrola používá metodu `grow(1).intersects(0)` což znamená, že se testuje nejen skutečný průnik obdélníků, ale i jejich rozšíření o jednu dlaždici na každou stranu. V praxi tak mezi dvěma místnostmi vždy zůstává alespoň jednopolíčková mezera. Pokud by se nová místnost překrývala s některou z již existujících, algoritmus ji zahodí a pokračuje dalším pokusem.

V případě, že nová místnost překryv nevykazuje, algoritmus ji přidá do seznamu místností a zapíše její vnitřní plochu do mapy jako podlahu. Dvojice vnořených cyklů projde všechny buňky uvnitř obdélníku místnosti a nastaví jejich hodnotu na `TILE_FLOOR`.

Každá nová místnost je následně propojena chodbou s předchozí místností v pořadí umístění. První místnost je z tohoto propojování vyloučena, protože v okamžiku jejího přidání ještě neexistuje žádná jiná místnost. Pro propojení se využívají středy obou místností získané pomocí metody `get_center()`. Vlastní vytvoření chodby zajišťuje pomocná funkce `_carve_corridor()`, která mezi dvěma zadanými body vyznačí chodbu tvaru písmene L. Algoritmus náhodně rozhoduje, zda bude chodba nejprve vedena ve vodorovném a poté ve svislém směru, nebo opačně. Díky tomu výsledné mapy nepůsobí monotónně.

Chodba se neskládá pouze z jedné dlaždice, ale má nastavitelnou šířku. Implementace nastavuje výchozí hodnotu parametru `corridor_width` na 2. Výsledná chodba je tedy široká 2 dlaždice a při zápisu dlaždic chodby algoritmus kontroluje, zda se zapisované souřadnice nacházejí uvnitř hranic mapy, a to pomocí pomocné funkce `_in_bounds()`.



Obrázek 3: Ukázka mapy generované pomocí algoritmu náhodných místností a chodeb (45×45)

3.3.2 Binary Space Partitioning

Implementace metody BSP se nachází v souboru `bsp_generator.gd`.

Pomocná třída `Branch` reprezentuje stromovou strukturu mapy. Každý uzel uchovává svou pozici (`Vector2i`), velikost (`Vector2i`) a náhodné odsazení `padding` typu `Vector4i`, jehož složky udávají vzdálenost místnosti od okraje oblasti, náhodně 2 až 3 dlaždice na každé straně. Díky tomuto odsazení nevznikají místnosti těsně u hranic oblastí a mezi nimi zůstává dostatečný prostor.

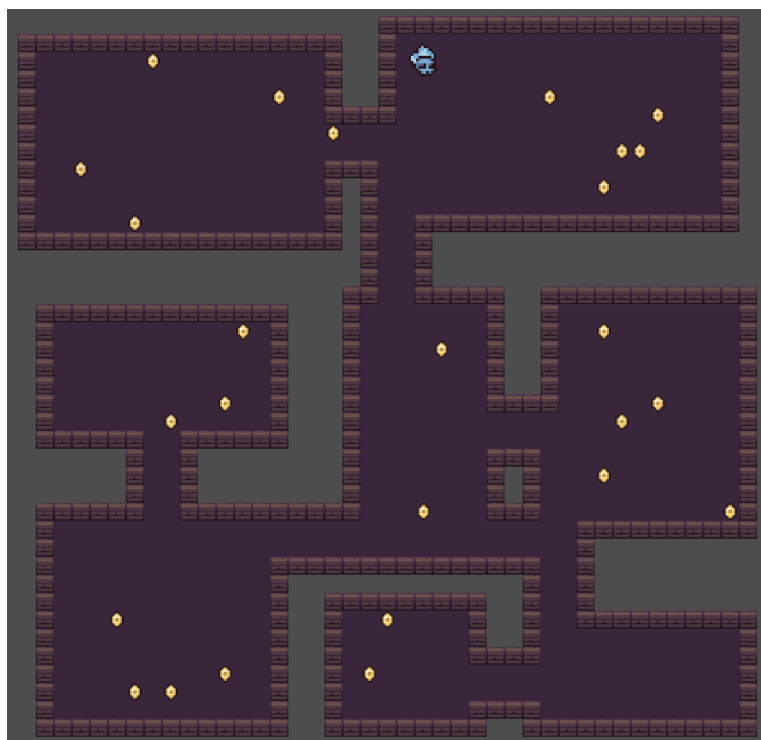
Kořenový uzel pokrývá celou mapu zmenšenou o jednu dlaždici na každé straně, tedy oblast od pozice (1,1) o velikosti $(width - 2) \times (height - 2)$. Tím zůstane na okraji mapy jednopolíčkový rámeček.

Rekurzivní dělení prostoru provádí metoda `split(min_size, rng)`. Algoritmus určuje směr řezu deterministicky. Pokud je výška oblasti větší nebo rovna šířce (`size.y >= size.x`), dělí se horizontálně, jinak vertikálně. Algoritmus ukončí dělení, pokud by výsledné podoblasti byly menší než `min_size × 2`. Algoritmus volí pozici řezu náhodně v rozmezí 35–65 % délky dělené části, takže vzniklé podoblasti nemusí být stejně velké.

Generátor volá funkci `split()` s hodnotou konstanty `MIN_SIZE = 13` a rekurzivně dělí prostor tak dlouho, dokud to velikost oblasti umožňuje. Počet výsledných listových uzlů tedy není fixní, ale závisí na rozměrech mapy. Na větších mapách vzniká více oblastí a tím i více místností. V každém listu algoritmus

vykreslí místnost odpovídající ploše uzlu zmenšené o jeho odsazení `padding`.

Teprve po dokončení celého stromu metoda `get_corridor_pairs()` vrátí chodby. Prochází strom rekurzivně a pro každý vnitřní uzel vybere první listový uzel z levého a pravého podstromu a uloží dvojici jejich středů. Metoda `get_room_center()` vypočítá středy místností se zohledněním odsazení `padding`, takže výsledný bod leží vždy uvnitř skutečné průchozí plochy místnosti, nikoliv jen v geometrickém středu celé podoblasti. Každá chodba má tvar písmene L, nejprve vede ve vodorovném, a poté ve svislém směru. Chodba je široká 2 dlaždice.



Obrázek 4: Ukázka mapy generované pomocí algoritmu BSP (45×45)

3.3.3 Metoda Náhodné procházky

Metoda Náhodné procházky se nachází v souboru `drunkards_generator.gd`. Algoritmus začíná ve středu mapy na pozici $\text{width}/2, \text{height}/2$. Algoritmus ihned vytvoří kolem počáteční pozice oblast podlahy o velikosti 3×3 dlaždic pomocí funkce `_carve_3x3()`. Cílový počet podlahových dlaždic činí 50 % celkové plochy mapy podle vzorce (1), kde konstanta `TARGET_COVERAGE` má hodnotu 50.

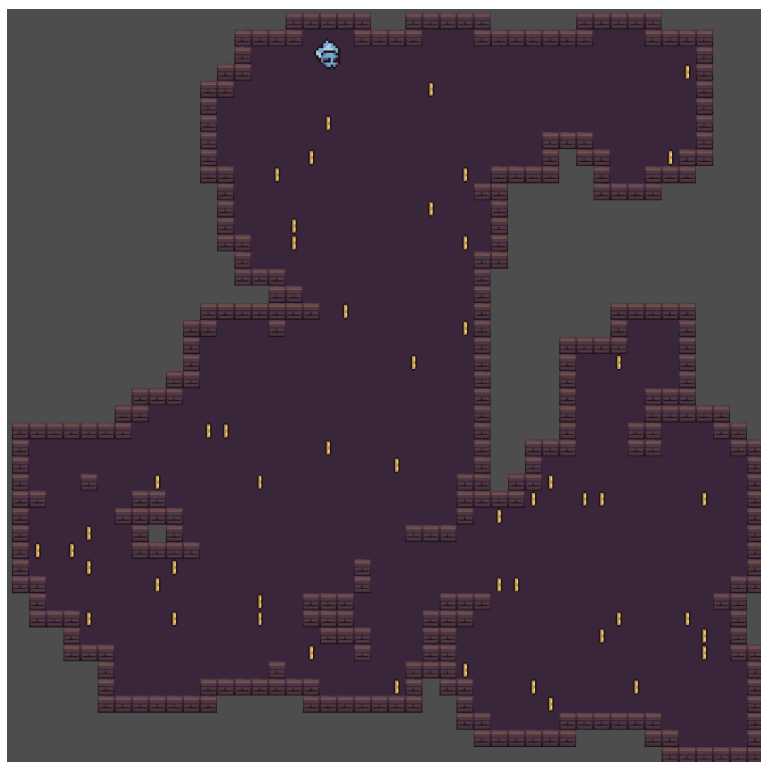
$$\text{cílový_počet} = \frac{\text{width} \times \text{height} \times \text{TARGET_COVERAGE}}{100} \quad (1)$$

Algoritmus běží v cyklu, který pokračuje, dokud počet podlahových dlaždic nedosáhne cílového počtu. V každém kroku algoritmus náhodně zvolí jeden ze

čtyř směrů (nahoru, dolů, doleva, doprava) a posune pozici generátoru o jednu dlaždici. Funkce `clamp()` ji omezuje tak, aby zůstávala alespoň 2 dlaždice od okraje mapy. Tato rezerva zajistí, že i při vykreslování oblastí 3×3 nepřekročí hranice pole.

Algoritmus rozlišuje dva typy vykreslování. Každých 15 až 25 kroků (náhodně volený interval) algoritmus vytvoří na aktuální pozici oblast 3×3 dlaždic, která představuje malou místnost. V ostatních krocích vykreslí pouze oblast 2×2 dlaždic, což odpovídá chodbovému úseku. Obě varianty zajišťuje jediná pomocná funkce `_carve_area()`, které se předá požadovaná velikost oblasti (3 nebo 2). Funkce zapisuje podlahu pouze tam, kde dosud podlaha nebyla, a vrací počet nově vytvořených dlaždic, aby mohl být průběžně aktualizován celkový počet.

Průchodnost celé mapy je zaručena tím, že veškerý průchozí prostor vzniká z jedné souvislé procházky. Díky pevně stanovenému cílovému podílu podlahy je pokrytí mapy pro danou velikost relativně stabilní. Variabilita se projevuje především ve tvaru a rozmístění vytvořených prostorů.



Obrázek 5: Ukázka mapy generované pomocí algoritmu Náhodné procházky (45×45)

3.3.4 Buněčné automaty

Implementace Buněčných automatů je v souboru `cellular_generator.gd`.

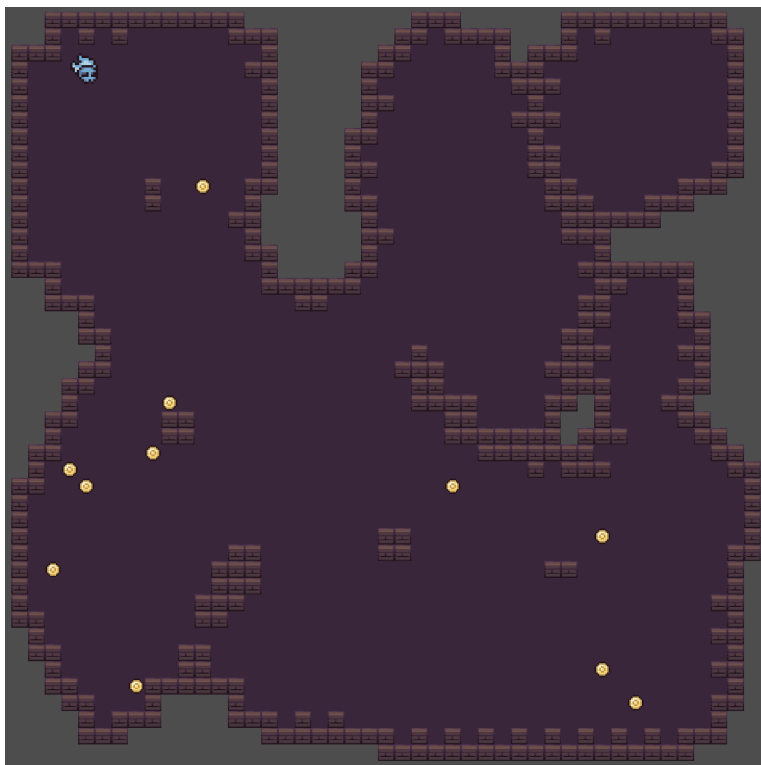
V první fázi algoritmus náhodně nastaví každou vnitřní buňku mapy na podlahu s pravděpodobností `FILL_PROBABILITY = 0.45`, tedy 45%. Buňky na

okraji mapy (první a poslední řádek a sloupec) zůstávají vždy prázdné, čímž vznikne pevný rámec, který zabraňuje rozšíření jeskynních prostor až k hranicím pole.

Algoritmus provede celkem 5 simulačních kroků (konstanta `SIMULATION_STEPS`). V každém kroku algoritmus vytvoří novou mapu, aby se průběžné změny nepromítaly do výpočtu sousedů v témže kroku. Pro každou buňku funkce `_count_floor_neighbo` spočítá počet podlahových sousedů v Mooreově okolí (osm přilehlých buněk). Pokud je tento počet alespoň 4, buňka se stane podlahou, v opačném případě zůstane prázdná. Okrajové buňky ve všech krocích zůstávají prázdné.

Opakováním tohoto pravidla se náhodný šum postupně vyhladí do organických tvarů připomínajících jeskynní systémy. Volba počáteční pravděpodobnosti 45 % a prahu 4 sousedů představuje kompromis mezi dostatečně otevřenými prostory a přirozeně působícími stěnami. Při vyšší hodnotě by vznikaly příliš velké otevřené části, při nižší by byly oblasti méně průchozí, protože by velká část buněk tvořila stěny.

U map generovaných touto metodou se mohou vyskytovat izolované průchozí oblasti, protože algoritmus nezaručuje souvislost prostoru. Z tohoto důvodu bylo při umisťování hráče nutné zohlednit průchozí plochu a upřednostnit umístění do největší spojitě oblasti mapy, což implementace nakonec zahrnula do všech metod.



Obrázek 6: Ukázka mapy generované pomocí algoritmu Buněčných automatů (45×45)

3.3.5 Metoda Bludiště

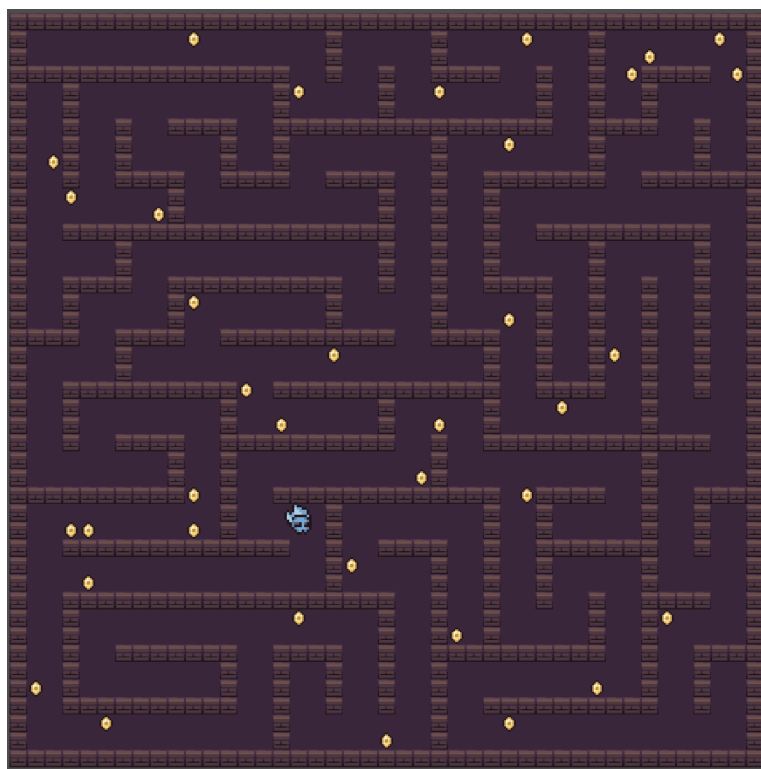
Implementace Bludiště se nachází v souboru `maze_generator.gd`.

Implementace metody vychází z algoritmu *recursive backtracker* (prohledávání do hloubky se zásobníkem). Algoritmus nejprve rozdělí mapu na mřížku abstraktních buněk. Každá buňka zabírá 2×2 dlaždic podlahy a mezi sousedními buňkami je jednodlaždicová mezera, takže na jednu buňku připadá úsek 3×3 dlaždic. Počet buněk závisí na rozměrech mapy, kde odečtení 2 zohledňuje jednodlaždicový okraj mapy na obou stranách:

$$\text{cells_w} = \max\left(1, \left\lfloor \frac{\text{width} - 2}{3} \right\rfloor\right), \quad \text{cells_h} = \max\left(1, \left\lfloor \frac{\text{height} - 2}{3} \right\rfloor\right). \quad (2)$$

Stav každé buňky představuje bitová maska čtyř stěn ($N = 1$, $E = 2$, $S = 4$, $W = 8$), na začátku jsou všechny stěny přítomné. Algoritmus startuje v buňce $(0, 0)$ a v každém kroku náhodně vybere nenavštíveného souseda. Ze společné stěny algoritmus odstraní příslušný bit na obou stranách, například při pohybu na sever se z aktuální buňky odstraní bit N a ze sousední buňky se odstraní bit S . Pokud aktuální buňka nemá žádného nenavštíveného souseda, algoritmus se vrací zpět po zásobníku. Průchod končí, jakmile algoritmus navštíví všechny buňky.

Výsledkem je takzvané perfektní bludiště, ve kterém mezi libovolnými dvěma buňkami existuje právě jedna cesta. Po dokončení prohledávání algoritmus převede buněčnou reprezentaci na dlaždicovou mapu. Každou buňku vykreslí jako oblast 2×2 podlahových dlaždic na pozici $(1 + cx \cdot 3, 1 + cy \cdot 3)$. Tam, kde algoritmus stěnu odstranil, jednodlaždicovou mezeru mezi sousedními buňkami rovněž vyplní podlahou o šířce 2 dlaždic, čímž vznikne průchozí koridor.



Obrázek 7: Ukázka mapy generované pomocí algoritmu Bludiště (45×45)

3.4 Metodika měření a vyhodnocení

Benchmarkový režim spouští všechny algoritmy pro pět velikostí map: 20×50 , 60×60 , 70×30 , 100×100 a 150×150 dlaždic. Tyto velikosti pokrývají jak malé, tak velké rozměry a umožňují posoudit vliv tvaru mapy (čtverec versus obdélník) na chování algoritmů. Pro každou kombinaci algoritmu a velikosti mapy generování proběhne opakovaně v deseti nezávislých bězích. Benchmark proběhl celkem $5 \times$, čímž vzniklo celkem 1250 záznamů.

Funkce `Time.get_ticks_usec()` zajišťuje měření času s mikrosekundovou přesností. Měří se výhradně čas funkce `generate(width, height)` příslušného algoritmu. Vizualizace mapy, vykreslování dlaždic, umisťování hráče a další operace nejsou součástí měřeného intervalu. Výsledný čas se převede na milisekundy.

Každý běh zahrnuje následující metriky:

- **čas generování** (`gen_time_ms`) – doba trvání samotného algoritmu v milisekundách
- **počet podlahových dlaždic** (`floor_tiles`) – celkový počet podlahových dlaždic ve výsledné mapě
- **pokrytí** (`coverage`) – podíl podlahových dlaždic z celkového počtu bu-

něk mapy, vyjádřený v procentech:

$$\text{coverage} = \frac{\text{floor_tiles}}{\text{width} \times \text{height}} \times 100 \quad (3)$$

Aplikace průběžně ukládá naměřené hodnoty každého běhu do souboru `results.csv`. Každý záznam obsahuje název algoritmu, rozměry mapy, pořadí běhu, čas generování, pokrytí a počet podlahových dlaždic.

Naměřená data vyhodnocuje samostatný skript `result_analyser.py` v jazyce Python s využitím knihoven `pandas` a `matplotlib`. Skript načte soubor `results.csv` a pro každou kombinaci algoritmu a velikosti mapy vypočítá souhrnné statistiky – průměr, směrodatnou odchylku, minimum a maximum času generování i pokrytí. Skript tyto statistiky uloží do souboru `results_summary.csv`. Následně skript automaticky vygeneruje sadu grafů: sloupcové grafy průměrného času a pokrytí pro jednotlivé algoritmy, škálovací křivky v závislosti na velikosti mapy, boxploty rozložení hodnot a bodový graf kompromisu mezi rychlostí a pokrytím. Všechny grafické výstupy použité v následující kapitole tento skript vygeneroval.

4 Výsledky

Všechny grafy v této kapitole vznikly z dat naměřených benchmarkovým režimem popsáným v předchozí podkapitole. Osy a legendy grafů jsou uvedené v anglickém jazyce. Názvy algoritmů v grafech odpovídají anglickým označením *Rooms* (Místnosti), *BSP*, *Drunkard's* (Náhodná procházka), *Cellular* (Buněčné automaty) a *Maze* (Bludiště). Každý popis obrázku uvádí český překlad nadpisu grafu.

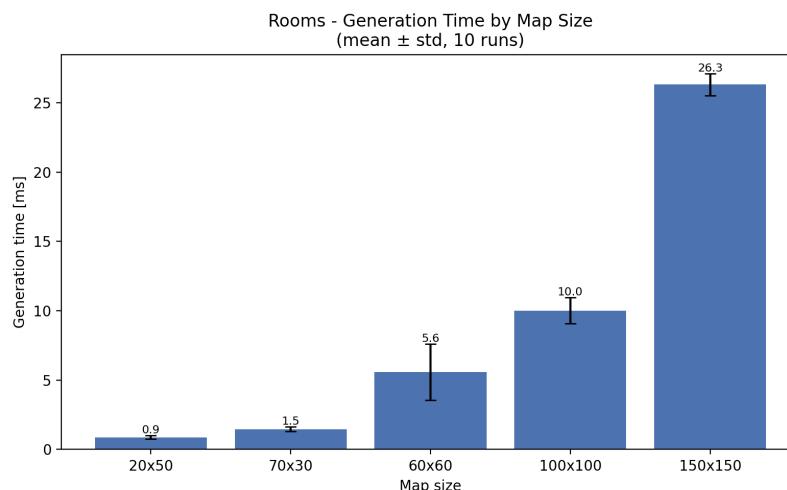
4.1 Výsledky jednotlivých algoritmů

4.1.1 Místnosti a chodby

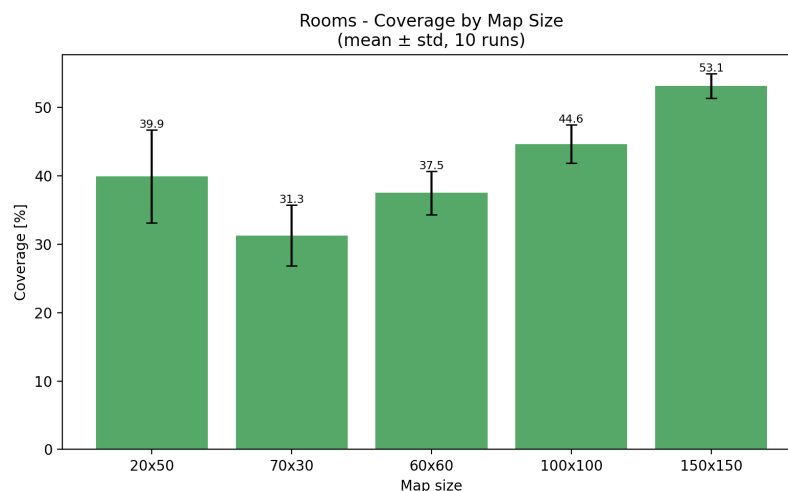
Algoritmus náhodných místností a chodeb vykazuje nízké časy generování na menších mapách. Na mapě 20×50 dosahuje průměrný čas přibližně 0,88 ms. S rostoucí velikostí mapy čas narůstá výrazněji. Na mapě 150×150 dosahuje průměrný čas přibližně 26,3 ms.

Pokrytí mapy průchozím prostorem roste s velikostí mapy. Na menších mapách (20×50 , 70×30) se pohybuje v rozmezí 31–41 %, na mapě 100×100 činí přibližně 44 % a na mapě 150×150 dosahuje přibližně 53 %.

Variabilita mezi jednotlivými běhy je u tohoto algoritmu patrná jak v čase generování, tak v pokrytí. Pokrytí se mění podle velikosti a rozmístění náhodně vygenerovaných místností. Na menších mapách, kde vzniká minimálně pět místností, je variabilita pokrytí vyšší.



Obrázek 8: Místnosti a chodby – průměrný čas generování podle velikosti mapy



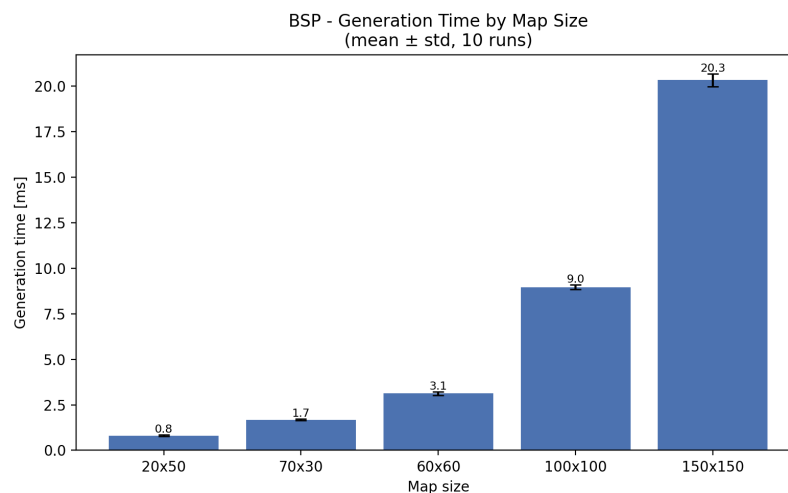
Obrázek 9: Místnosti a chodby – průměrné pokrytí podle velikosti mapy

4.1.2 Binary Space Partitioning

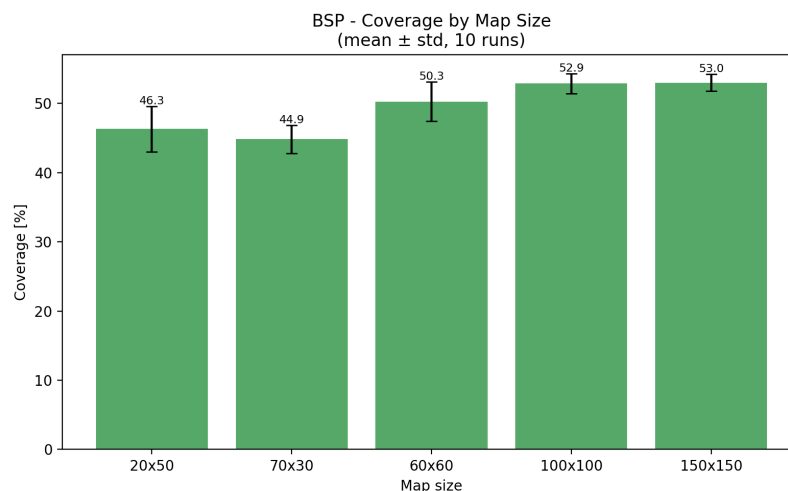
Algoritmus BSP vytváří mapy s dobře definovanými místnostmi a systematickým propojením. Čas generování roste s velikostí map, od přibližně 0,81 ms na mapě 20×50 po 20,3 ms na mapě 150×150.

Pokrytí mapy u BSP roste s velikostí mapy. Na nejmenší mapě 20×50 činí pokrytí přibližně 46,3 %, na mapě 150×150 dosahuje hodnot kolem 53,0 %.

Variabilita času mezi běhy je u BSP nízká, směrodatné odchylky jsou malé, což odpovídá determinističtější povaze rekurzivního dělení prostoru. Variabilita pokrytí je střední, výrazně totiž závisí na náhodném průběhu dělení, který ovlivňuje počet a velikost vzniklých místností.



Obrázek 10: BSP – průměrný čas generování podle velikosti mapy



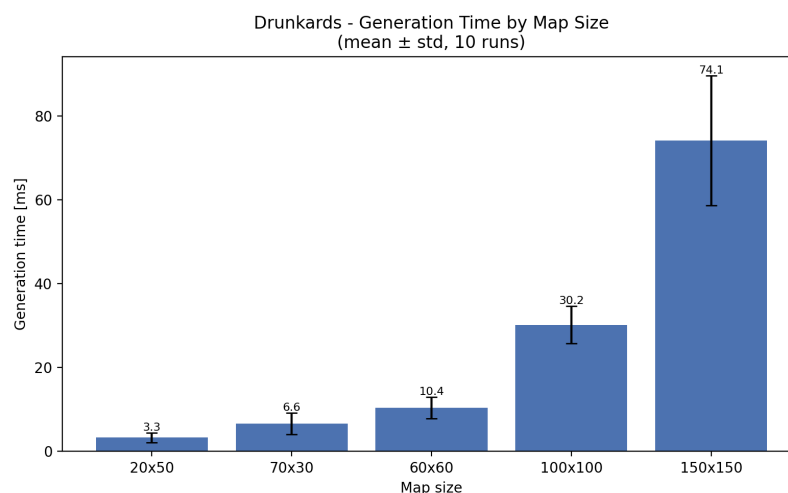
Obrázek 11: BSP – průměrné pokrytí podle velikosti mapy

4.1.3 Náhodná procházka

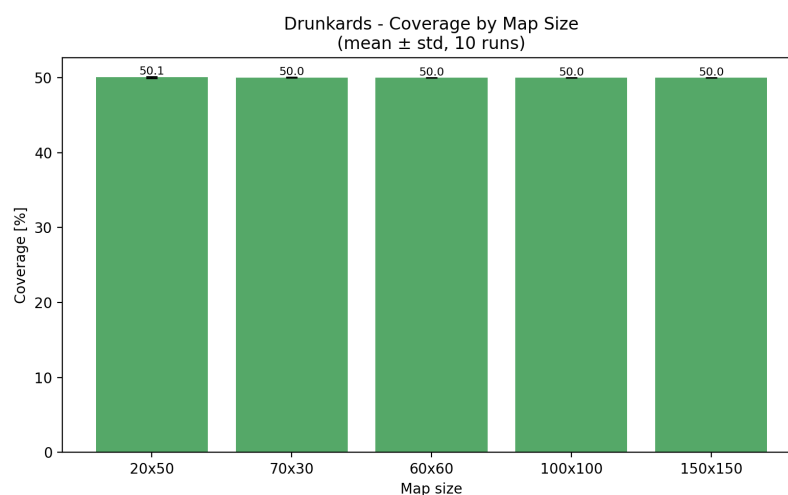
Algoritmus Náhodné procházky se od ostatních metod odlišuje především konstantním pokrytím mapy. Výsledné pokrytí se ve všech testovaných konfiguracích pohybuje v rozmezí 50,0–50,1 %, což přímo odpovídá implementaci, ve které je cílový počet podlahových dlaždic stanoven pevně. Variabilita pokrytí je tedy prakticky nulová.

Čas generování vykazuje výraznější variabilitu mezi jednotlivými běhy. Na mapě 20×50 se průměrný čas pohybuje kolem 3,3 ms. Na největší mapě 150×150 dosahuje přibližně 74,1 ms. Hodnota 50 % je přitom nastavitelný parametr a změnou této konstanty lze dosáhnout libovolného cílového pokrytí.

Náhodná procházka tak představuje algoritmus s předvídatelným pokrytím, avšak výrazně proměnlivým časem generování. Délka výpočtu závisí na tom, jak efektivně procházka pokrývá nové oblasti.



Obrázek 12: Náhodná procházka – průměrný čas generování podle velikosti mapy



Obrázek 13: Náhodná procházka – průměrné pokrytí podle velikosti mapy

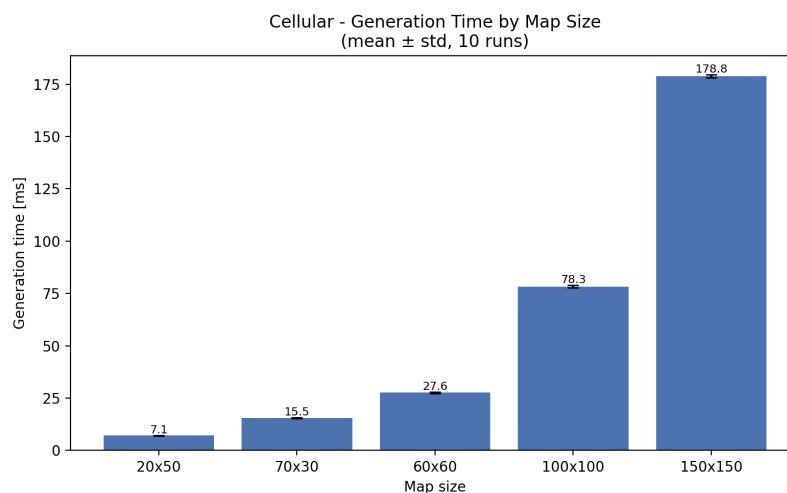
4.1.4 Buněčné automaty

Buněčné automaty jsou z testovaných algoritmů nejpomalejší. Na nejmenší mapě 20×50 trvá generování přibližně 7,1 ms, na mapě 100×100 již 78,3 ms a na největší mapě 150×150 dosahuje průměrný čas přibližně 178,8 ms. Hlavní příčinou je algoritmická náročnost. Algoritmus provádí 5 úplných průchodů přes všechny buňky mapy a pro každou buňku vyhodnocuje 8 okolních sousedů. To představuje řádově více operací než u ostatních algoritmů. Naměřené absolutní časy odrážejí také fakt, že GDScript je interpretovaný jazyk; primární příčinou výrazného rozdílu oproti ostatním metodám zůstává samotná struktura algoritmu.

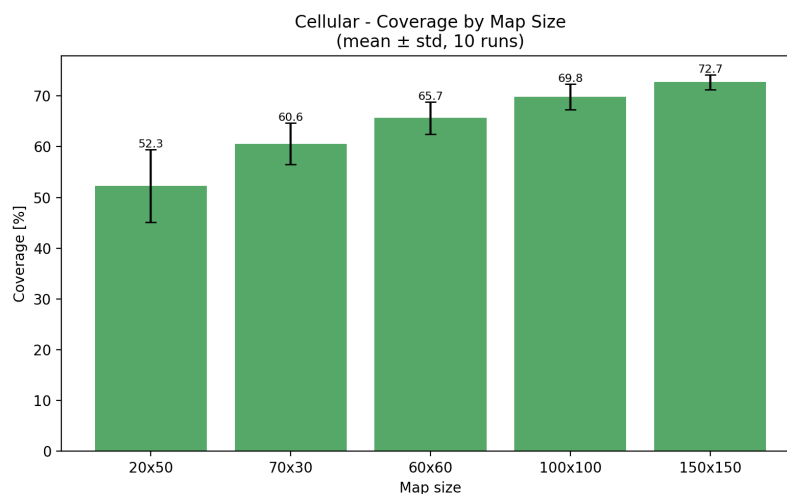
Pokrytí mapy u Buněčných automatů dosahuje vysokých hodnot a s rostoucí velikostí mapy mírně roste, od přibližně 52 % na mapě 20×50 po 72,7 % na

mapě 150×150. Variabilita pokrytí je přitom nezanedbatelná, zejména na menších mapách, kde náhodná inicializace má větší vliv na výsledný tvar průchozích oblastí.

Čas generování je mezi běhy prakticky konstantní a směrodatné odchylky jsou velmi malé. Celkový počet operací tak závisí primárně na velikosti mapy.



Obrázek 14: Buněčné automaty – průměrný čas generování podle velikosti mapy



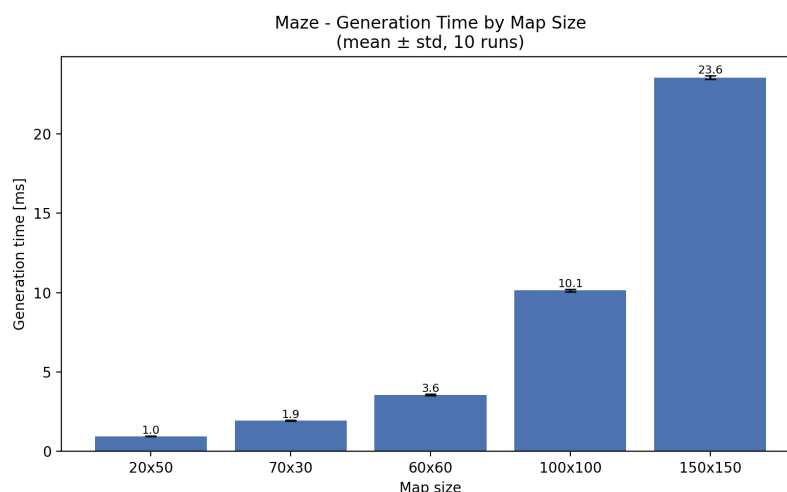
Obrázek 15: Buněčné automaty – průměrné pokrytí podle velikosti mapy

4.1.5 Bludiště

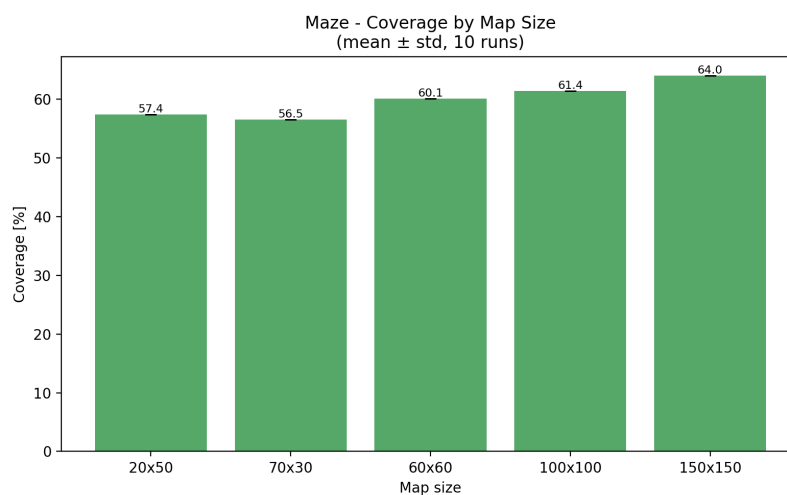
Algoritmus generování Bludiště se vyznačuje zcela deterministickým pokrytím mapy. Hodnota pokrytí je pro danou velikost mapy vždy shodná bez ohledu na konkrétní běh. Na mapě 20×50 činí pokrytí 57,4 %, na mapě 150×150 pak 64,0 %. Nulová variabilita pokrytí vyplývá z principu algoritmu recursive backtracker,

který vždy navštíví všechny buňky mřížky a výsledný poměr podlahových dlaždic závisí pouze na rozměrech mapy a velikosti buněk.

Čas generování je u tohoto algoritmu srovnatelný s BSP. Na nejmenší mapě činí přibližně 0,96 ms, na největší pak 23,6 ms. Variabilita času mezi běhy je velmi nízká, protože algoritmus vždy provede stejný počet kroků.



Obrázek 16: Bludiště – průměrný čas generování podle velikosti mapy



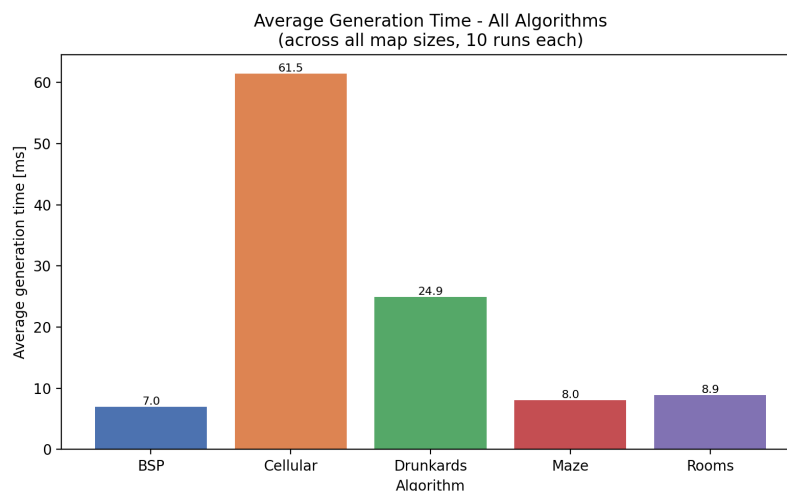
Obrázek 17: Bludiště – průměrné pokrytí podle velikosti mapy

4.2 Srovnání algoritmů

Zobrazené hodnoty jsou průměry přes všechny testované velikosti map, pokud není uvedeno jinak (50 měření na každou kombinaci algoritmu a velikosti, celkem 250 měření na algoritmus, cyklus proběhl celkem 5×). Konkrétní hodnoty pro jednotlivé velikosti shrnuje tabulka 3.

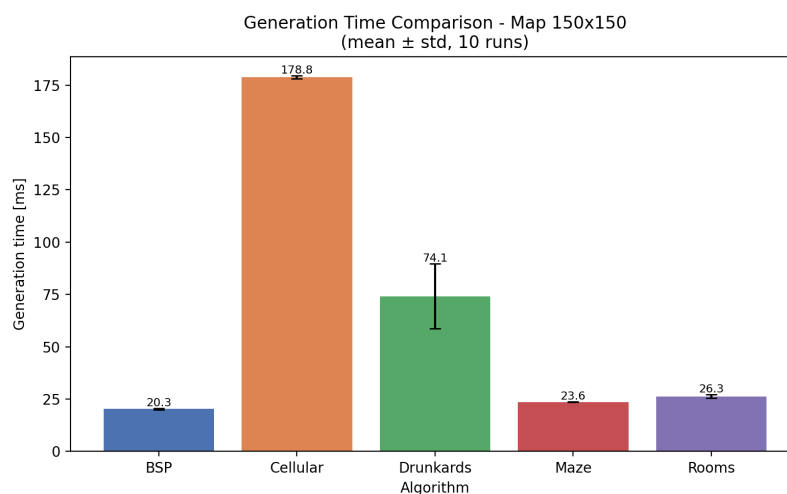
4.2.1 Průměrný čas generování

Graf 18 zobrazuje průměrný čas generování algoritmů přes všechny testované velikosti map. Mezi algoritmy jsou výrazné rozdíly. Nejrychlejšími metodami jsou BSP a Bludiště, jejichž celkový průměr se pohybuje kolem 7–8 ms. Místnosti dosahují srovnatelného průměru. Náhodná procházka je výrazně pomalejší s celkovým průměrem přibližně 25 ms. Buněčné automaty jsou řádově nejpomalejší, jejich celkový průměr přesahuje 61 ms, tedy přibližně osmkrát více než u BSP.



Obrázek 18: Srovnání průměrného času generování všech algoritmů

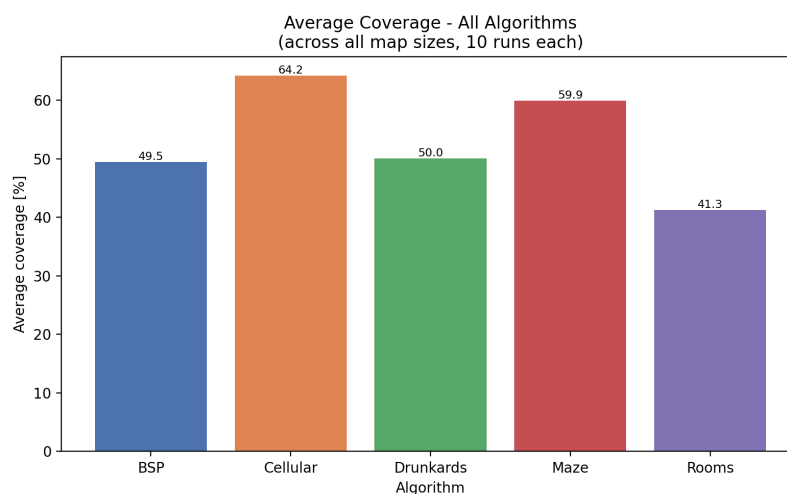
Na největší testované mapě 150×150 jsou absolutní rozdíly doby generování ještě výraznější: BSP dosahuje 20,3 ms, Bludiště 23,6 ms, Místnosti 26,3 ms, Náhodná procházka 74,1 ms a Buněčné automaty 178,8 ms (graf 19).



Obrázek 19: Srovnání průměrného času generování na mapě 150×150

4.2.2 Průměrné pokrytí

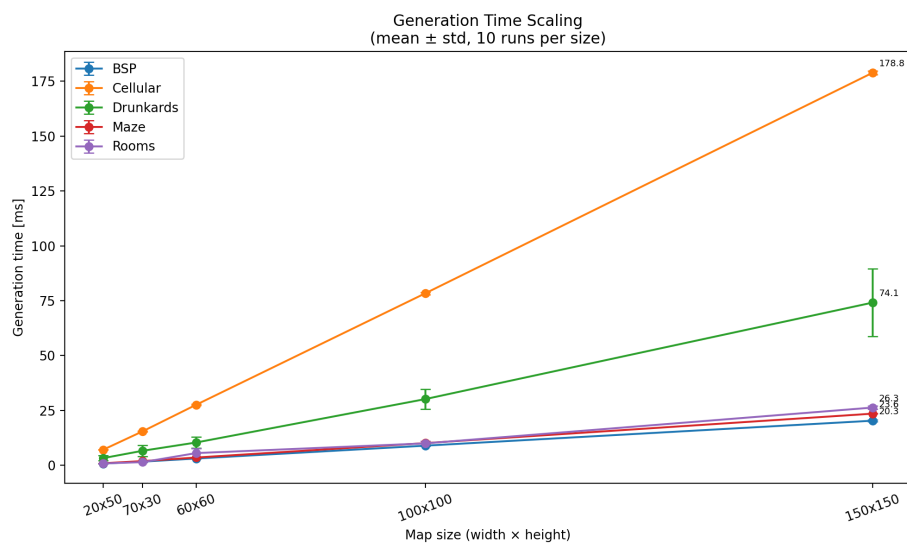
Graf 20 zobrazuje průměrné pokrytí mapy průchozím prostorem přes všechny testované velikosti. Nejvyššího celkového pokrytí dosahují Buněčné automaty (přibližně 64 %), následované algoritmem Bludiště s pokrytím kolem 60 %. BSP dosahuje celkového průměru téměř 50 %, Náhodná procházka má konstrukčně dané pokrytí 50 %, které se s velikostí mapy nemění. Místnosti a chodby dosahují nejnižšího celkového průměru, a to přibližně 42 %, avšak pokrytí výrazně roste s velikostí mapy (viz graf 22).



Obrázek 20: Srovnání průměrného pokrytí všech algoritmů

4.2.3 Škálování s velikostí mapy

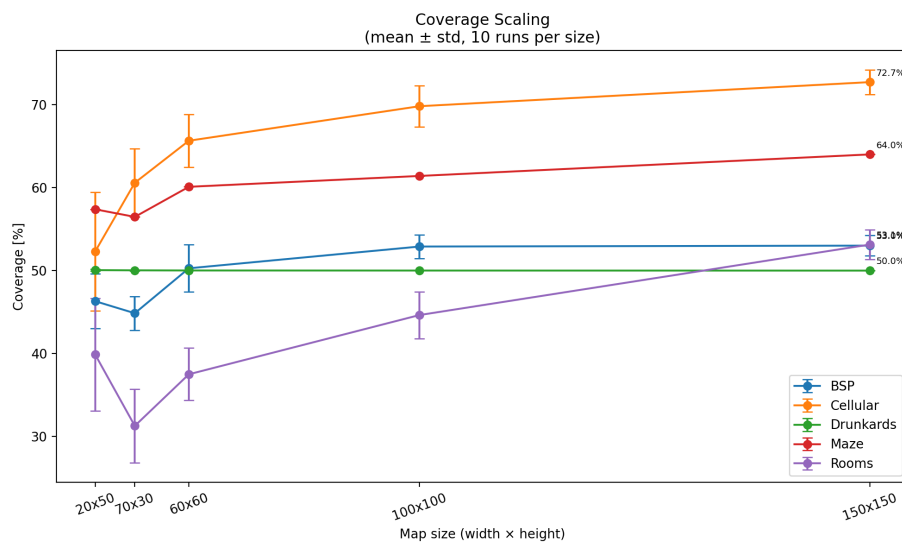
Graf 21 znázorňuje, jak se čas generování mění s rostoucím počtem buněk mapy. BSP a Bludiště vykazují přibližně lineární nárůst času. Algoritmus Místností roste rovněž přibližně lineárně, avšak s vyšší konstantou, protože na velkých mapách generuje výrazně více místností. Náhodná procházka roste rychleji, a to kvůli opakovanému navracení do již odkrytých oblastí. Buněčné automaty vykazují nejstrmější nárůst.



Obrázek 21: Škálování času generování s počtem buněk mapy

Graf 22 ukazuje vývoj pokrytí v závislosti na velikosti mapy. BSP, Buněčné automaty a Místnosti vykazují rostoucí tendenci pokrytí s velikostí mapy. Bludiště má mírně rostoucí, ale velmi stabilní pokrytí. Náhodná procházka zůstává konstantní na hodnotě 50 %.

Výjimku tvoří mapa 70×30, u které pokrytí algoritmů Místností, BSP a Bludiště klesá pod hodnoty dosažené na mapě 20×50, přestože celkový počet buněk je větší (2 100 vs. 1 000).

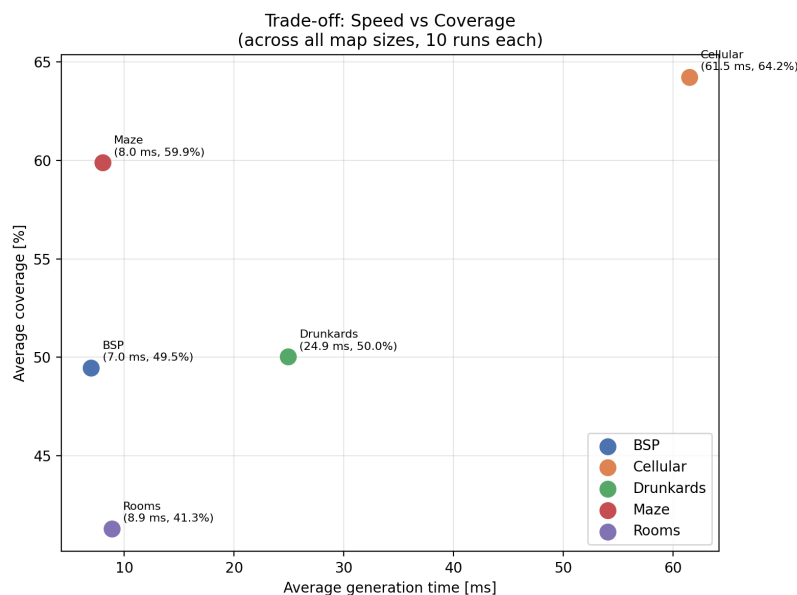


Obrázek 22: Škálování pokrytí s počtem buněk mapy

4.2.4 Kompromis mezi rychlostí a pokrytím

Graf 23 vizualizuje vztah mezi průměrným časem generování a průměrným pokrytím pro jednotlivé algoritmy (oba průměry přes všechny testované velikosti map). Ideální algoritmus by se nacházel v levé horní části grafu, tedy s nízkým časem a vysokým pokrytím.

BSP a Bludiště se nacházejí v oblasti nízkého času a středně vysokého pokrytí; Bludiště dosahuje vyššího pokrytí (kolem 60 %) než BSP (kolem 51 %). Místnosti nabízí srovnatelný čas s nižším celkovým pokrytím. Buněčné automaty dosahují nejvyššího pokrytí ze všech metod, avšak za cenu výrazně vyššího výpočetního času. Náhodná procházka představuje předvídatelné pokrytí 50 %, avšak s vyšším časem generování než BSP, Bludiště a Místnosti.

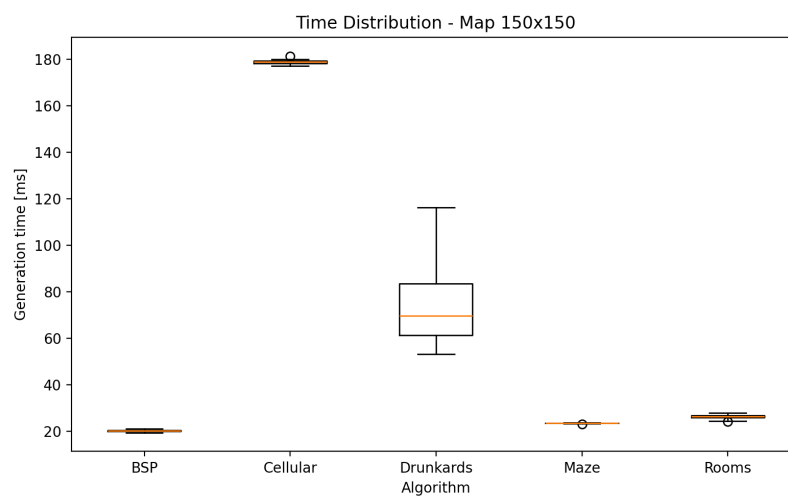


Obrázek 23: Kompromis mezi rychlostí a pokrytím

4.2.5 Rozptyl výsledků na velké mapě

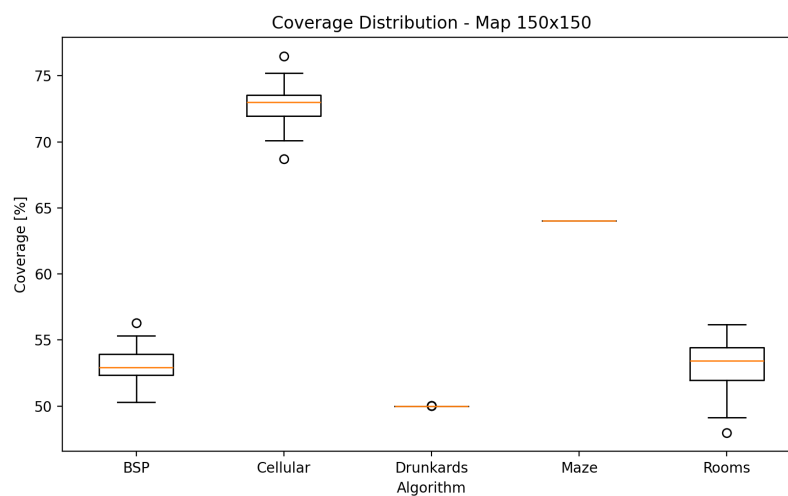
Variabilitu času a pokrytí na největší testované mapě 150×150 zobrazují box-ploty.

Graf 24 ukazuje, že BSP vykazuje nejmenší relativní rozptyl času, zatímco Náhodná procházka má nejvyšší variabilitu.



Obrázek 24: Rozptyl času generování na mapě 150×150

Obrázek 25 ukazuje, že Bludiště a Náhodná procházka mají prakticky nulovou variabilitu pokrytí, zatímco Místnosti, BSP a Buněčné automaty vykazují větší rozptyl.



Obrázek 25: Rozptyl pokrytí na mapě 150×150

4.3 Souhrnná tabulka výsledků

Tabulka 3 obsahuje průměrné hodnoty času generování a pokrytí mapy pro všechny testované kombinace algoritmů a velikostí map.

Algoritmus	20×50	60×60	70×30	100×100	150×150
<i>Průměrný čas generování [ms]</i>					
Rooms	0,88	5,59	1,46	10,01	26,33
BSP	0,81	3,12	1,69	8,97	20,34
Drunkard's	3,29	10,41	6,63	30,18	74,14
Cellular	7,10	27,60	15,47	78,34	178,83
Maze	0,96	3,57	1,95	10,14	23,55
<i>Průměrné pokrytí [%]</i>					
Rooms	39,88	37,49	31,27	44,64	53,14
BSP	46,31	50,28	44,86	52,90	53,01
Drunkard's	50,06	50,02	50,03	50,01	50,00
Cellular	52,31	65,67	60,60	69,83	72,74
Maze	57,40	60,11	56,48	61,42	64,02

Tabulka 3: Průměrný čas generování a pokrytí

4.4 Shrnutí výsledků

Algoritmy se podle výpočetní náročnosti rozdělují do tří skupin. Do první skupiny patří BSP, Bludiště a Místnosti, jejichž časy generování na mapě 150×150 nepřekračují 27 ms. Náhodná procházka tvoří střední kategorii s průměrným časem přibližně 74,1 ms. Buněčné automaty jsou nejpomalejší s průměrným časem přibližně 178,8 ms.

Z hlediska pokrytí mapy se algoritmy rovněž výrazně liší. Buněčné automaty dosahují na velkých mapách nejvyššího pokrytí přes 72 %. Bludiště dosahuje pokrytí v rozmezí 56–64 % v závislosti na velikosti mapy. BSP dosahuje relativně stabilního pokrytí v rozmezí 45–53 % napříč všemi velikostmi map. Místnosti dosahují na mapě 150×150 přibližně 53 %, avšak pokrytí výrazně stoupá s velikostí mapy (od přibližně 31 % na mapě 70×30). Náhodná procházka má dané pokrytí 50 %.

5 Diskuse

5.1 Silné a slabé stránky jednotlivých metod

Každý algoritmus vykazuje specifické silné a slabé stránky, které ovlivňují jeho vhodnost pro různé typy her.

Algoritmus BSP nabízí dobrou rovnováhu mezi rychlostí generování, pokrytím mapy a strukturovaností výsledku. Generované mapy obsahují jasně oddělené místnosti propojené chodbami. Nevýhodou je poměrně pravidelný vzhled výsledných map.

Algoritmus Místností po úpravě škálování počtu místností dosahuje na velkých mapách výrazně vyššího pokrytí a zároveň nabízí přirozenější rozložení prostoru. Variabilita mezi běhy je u tohoto algoritmu vyšší, což zvyšuje znovuhratelnost, ale zároveň snižuje předvídatelnost výsledku.

Buněčné automaty generují přirozeně působící jeskynní struktury, které se nejvíce odlišují od klasického dungeonového prostředí. Dosahují nejvyššího pokrytí ze všech testovaných metod. Hlavní nevýhodou je výrazně vyšší výpočetní náročnost a možný vznik izolovaných oblastí, které nejsou vzájemně průchozí. Důsledkem těchto izolovaných oblastí je, že výsledné pokrytí zahrnuje i nepřístupné části mapy. Skutečné přístupné pokrytí je proto nižší, než jakou hodnotu udává měřená metrika.

Náhodná procházka vytváří organicky tvarované chodby s předvídatelným pokrytím. Konstantní pokrytí může být výhodou v situacích, kdy je požadována přesná kontrola nad poměrem průchozího prostoru. Nevýhodou je nižší strukturovanost výsledného prostředí.

Implementace algoritmu Bludiště slouží primárně pro demonstraci odlišného přístupu k procedurálnímu generování. Generuje dokonalé bludiště s téměř nulovou variabilitou pokrytí a velmi nízkou variabilitou času. Z hlediska dungeonových map je však méně vhodný, protože vytváří převážně úzké chodby bez většího otevřeného prostoru.

Naměřené časy generování odpovídají teoretické složitosti jednotlivých algoritmů. Všechny implementované metody dosahují lineární složitosti $O(n)$, kde $n = w \cdot h$; rozdíly v rychlosti jsou způsobeny konstantními koeficienty. Buněčné automaty provádějí pro každou buňku $s = 5$ průchodů s vyhodnocením 8 sousedů ($\approx 40 \cdot n$ operací), což vysvětluje přibližně osminásobný časový rozdíl oproti BSP a Bludišti. Stochastická povaha Náhodné procházky způsobuje opakované navštěvování odkrytých oblastí a výraznou variabilitu doby generování, i přes lineární průměrnou složitost. Pokrytí mapy 70×30 klesá u algoritmů Místností, BSP a Bludiště pod hodnoty menší mapy 20×50 , protože nízký obdélník omezuje rozmístění místností i použitelné oblasti při dělení mřížky, čímž roste podíl stěn a okrajů vůči průchoznému prostoru.

5.2 Zkušenosti z implementace

U většiny metod zaručoval průchodnost mapy již princip algoritmu – explicitní propojování místností u Místností a BSP, návštěva všech buněk u Bludiště a spojitá chůze u Náhodné procházky. U Buněčných automatů však mohou po simulaci vzniknout oddělené nepřístupné kapsy. Umísťování hráče je přitom společné pro všechny metody: implementace nejprve pomocí BFS identifikovala největší souvislou oblast podlahových dlaždic a hráče umístila právě do ní. Jelikož je hráč reprezentován kolizním tělesem o velikosti 2×2 dlaždic, umístění vyžadovalo nalezení dlaždice s volným okolím 3×3 . Pokud taková dlaždice v největší souvislé oblasti neexistuje, implementace použije záložní pozici 2×2 .

Velikost hráče ovlivnila i návrh všech generovacích algoritmů. Chodby musely mít šířku alespoň 2 dlaždice, jinak by byla mapa pro hráče neprůchozí. Toto omezení se promítlo například do šířky L-chodeb u BSP a Místností, do vykreslování oblastí 2×2 v jednotlivých krocích Náhodné procházky a do velikosti buněk 2×2 u algoritmu Bludiště.

Úprava algoritmu Místností, při které pevný počet místností nahradilo škálování úměrné ploše mapy, vedla k výraznému zlepšení pokrytí na velkých mapách. Další významná úprava proběhla u BSP, kdy místo fixních 4 průchodů do hloubky algoritmus přešel na rekurzivní dělení, které pokračuje, dokud by výsledné podoblasti nebyly menší než `MIN_SIZE` $\times$ 2.

Ladění parametrů Buněčných automatů, konkrétně počtu simulačních kroků, mělo značný dopad na výsledek. Při 3 krocích zůstávaly v mapě četné izolované buňky, při 10 krocích se struktury výrazně vyhladily a pokrytí vzrostlo, avšak za cenu vyššího výpočetního času a velkého prázdného prostoru bez struktury. Hodnota 5 kroků představuje kompromis mezi kvalitou výsledných struktur a časovou náročností. Podobně parametr `TARGET_COVERAGE` u Náhodné procházky přímo určuje podíl průchozího prostoru, vyšší hodnota prodlužovala dobu generování a struktura byla opět velmi souvislá.

I drobná změna parametrizace může zásadně ovlivnit výsledné vlastnosti generovaného prostředí, jeho pokrytí, strukturu i čas generování.

5.3 Souvislost s předchozím výzkumem

Práce [3] rovněž porovnávala efektivitu PCG algoritmů pro generování 2D map a měřila čas generování a spotřebu výpočetních zdrojů kombinací algoritmů pro umísťování místností (Random Room Placement, BSP) a propojování chodbami (Random Point Connect, Drunkard's Walk, BSP Corridors). Oproti tomuto přístupu se předkládaná práce zaměřuje na pět samostatných generovacích metod (včetně Buněčných automatů a generování Bludiště) a jako metriku používá vedle času generování také pokrytí mapy průchozím prostorem. Obě práce se však shodují v závěru, že BSP patří mezi nejefektivnější přístupy z hlediska času generování.

5.4 Možnosti dalšího rozšíření práce

Prvním směrem rozšíření je zahrnutí strukturálních metrik. Pokrytí mapy průchozím prostorem je snadno měřitelná a objektivní metrika, avšak sama o sobě nevypovídá o strukturální kvalitě generovaného prostředí. Dvě mapy se stejným pokrytím se mohou zásadně lišit z hlediska hratelnosti. Například Bludiště a Buněčné automaty dosahují na mapě 60×60 podobného pokrytí (přibližně 60 a 66 %), přesto jsou výsledné struktury zcela odlišné: Bludiště vytváří spleitou síť úzkých chodeb, zatímco Buněčné automaty generují otevřené jeskynní prostory.

Relevantnějšími doplňkovými metrikami by mohly být například průměrná délka nejkratší cesty mezi dvěma náhodně vybranými průchozími dlaždicemi (měřitelná pomocí BFS), podíl slepých uliček nebo průměrný počet prostorově oddělených místností. Tyto metriky by umožnily kvantitativně zachytit rozdíly, které se v současném měření projevují pouze vizuálně, a lépe posoudit vhodnost jednotlivých metod pro konkrétní typy herního prostředí.

Dalším směrem je kombinace více metod v jedné mapě, například vyplnění BSP místností buněčnými automaty pro dosažení přirozenějšího vzhledu. Užitečným rozšířením by bylo také přidání sémantických pravidel pro rozmístění herních prvků, například zajištění, aby klíčové předměty ležely v dostatečné vzdálenosti od startu hráče, nebo adaptivní generování přizpůsobující parametry algoritmů postupu hráče hrou.

Závěr

Práce implementovala a experimentálně porovnála pět algoritmů procedurálního generování dungeonových map ve 2D hrách v herním enginu Godot.

V teoretické části práce popsala základní principy procedurálního generování obsahu, jeho historický vývoj a přehled vybraných metod včetně jejich výhod a omezení. Z řešeršní části práce vybrala pět algoritmů reprezentujících různé přístupy ke generování map: Náhodné umístění místností a chodeb, Binary Space Partitioning, Náhodná procházka (Drunkard's Walk), Buněčné automaty a generování Bludiště.

Práce implementovala všech pět algoritmů v herním enginu Godot s využitím skriptovacího jazyka GDScript. Systém vznikl v modulárním návrhu s unifikovaným rozhraním, které umožňuje snadné přidávání nových generovacích metod. Součástí aplikace je automatizovaný benchmarkový režim, který zajišťuje objektivní a opakovatelné měření výkonu jednotlivých algoritmů.

Experimentální měření proběhlo na pěti velikostech map v deseti nezávislých bězích pro každou konfiguraci, a to celkem $5 \times$. Výsledky ukázaly výrazné rozdíly mezi metodami. Algoritmy BSP a Bludiště se ukázaly jako nejrychlejší s časy kolem 20–24 ms na mapě 150×150 . Algoritmus Místností dosahuje pokrytí přibližně 53 % na největší mapě. Buněčné automaty generují mapy s nejvyšším pokrytím (přes 72 %), avšak s výrazně vyšším výpočetním časem, a to přibližně 178,8 ms na mapě 150×150 . Náhodná procházka poskytuje konstrukčně dané pokrytí 50 %, avšak s vyšším časem generování, přibližně 74 ms na největší mapě.

Práce ukázala, že volba generovací metody závisí na konkrétních požadavcích hry, protože každá metoda vytváří odlišný typ prostředí. Pro klasické dungeonové prostředí jsou vhodné algoritmy BSP a Místností, pro jeskynní struktury jsou ideální Buněčné automaty a Náhodná procházka a pro labyrinty algoritmus Bludiště.

Mezi hlavní omezení práce patří testování výhradně v prostředí Godot s jazykem GDScript a měření na jednom hardwarovém zařízení (MacBook Air 2025, M4, 24 GB RAM). Benchmarkové měření probíhalo bez fixovaného seedu, takže individuální výsledky konkrétního běhu nelze přesně reprodukovat. Statistické závěry jsou však podloženy 50 měřeními na každou konfiguraci.

Conclusions

This thesis implemented and experimentally compared five algorithms for procedural dungeon map generation in 2D games in the Godot game engine.

The theoretical part described the basic principles of procedural content generation, its historical development, and an overview of selected methods, including their advantages and limitations. From the literature review, the thesis selected five algorithms representing different approaches to map generation: Random room placement, Binary Space Partitioning, Drunkard’s walk, Cellular automata, and Maze generation.

All five algorithms were implemented in the Godot game engine using the GDScript scripting language. The system uses a modular design with a unified interface that allows easy addition of new generation methods. The application includes an automated benchmark mode that ensures objective and repeatable performance measurements.

Experimental measurements were conducted on five map sizes with ten independent runs per configuration, repeated five times in total. The results showed significant differences between the methods. BSP and Maze proved to be the fastest algorithms, with generation times around 20–24 ms on a 150×150 map. The Rooms algorithm achieves approximately 53 % coverage on the largest map. Cellular automata generate maps with the highest coverage (over 72 %), but at the cost of significantly higher computation time of approximately 178.8 ms on a 150×150 map. Drunkard’s walk provides a structurally fixed coverage of 50 %, but with a higher generation time of approximately 74 ms on the largest map.

The choice of generation method depends on the specific requirements of the game: BSP and Rooms are suitable for classic dungeon environments, cellular automata and drunkard’s walk for cave-like structures, and the Maze algorithm for labyrinthine layouts.

Limitations of this work include testing exclusively in the Godot environment with GDScript and measurements on a single hardware device (MacBook Air 2025, M4, 24 GB RAM). The benchmark measurements were conducted without a fixed random seed, meaning individual run results cannot be precisely reproduced; however, the statistical conclusions are supported by 50 measurements per configuration.

A Odkaz na repozitář projektu

Zdrojový kód aplikace je dostupný v repozitáři na platformě GitHub:

<https://github.com/Jim-23/ProceduralGenerationBP>

B Zkratky

2D Two-Dimensional

3D Three-Dimensional

ASCII American Standard Code for Information Interchange

BBC Micro British Broadcasting Corporation Microcomputer

BFS Breadth-first search

BSP Binary Space Partitioning

CSV Comma-Separated Values

DFS Depth-First Search

PCG Procedural Content Generation

RAM Random Access Memory

C Obsah elektronických dat

Elektronická data práce tvoří ZIP archiv celého projektu s následující strukturou:

src/

Zdrojové soubory projektu a `README.txt` s instrukcemi pro spuštění aplikace.

text/

Adresář s textem bakalářské práce.

Literatura

- [1] Shaker, Noor; Togelius, Julian; Nelson, Mark J. *Procedural Content Generation in Games: A Textbook and an Overview of Current Research*. 2016. 247 s. ISBN 978-3-319-42714-0.
- [2] Linden, Roland; Lopes, Ricardo; Bidarra, Rafael. Procedural Generation of Dungeons. *Computational Intelligence and AI in Games, IEEE Transactions on*. 2014, roč. 6, s. 78–89. Dostupný také z: <http://dx.doi.org/10.1109/TCIAIG.2013.2290371>.
- [3] Monaghan, Zina. *Comparing Procedural Content Generation Algorithms for Creating Levels in Video Games*. 2019. [Online; accessed 2026]. Dostupný také z: <https://arrow.tudublin.ie/cgi/viewcontent.cgi?article=1189&context=scschcomdis>.
- [4] Schuller, Dan. *Dive into beneath Apple Manor: A pre-rogue roguelike*. 2021. Dostupný z: <https://howtomakeanrpg.com/r/l/g/apple-manor.html>.
- [5] Wikipedia Contributors. *Beneath Apple Manor* — *Wikipedia, The Free Encyclopedia*. 2024. Dostupný z: https://en.wikipedia.org/w/index.php?title=Beneath_Apple_Manor&oldid=1260179812%22.
- [6] Procedural Content Generation Wiki. *Rogue* [online]. 2016 [cit. 2024-5-22]. Dostupný z: <http://pcg.wikidot.com/pcg-games:rogue>.
- [7] Wikipedia Contributors. *Rogue (video game)* — *Wikipedia, The Free Encyclopedia*. 2026. [Online; accessed 2026]. Dostupný z: [https://en.wikipedia.org/w/index.php?title=Rogue_\(video_game\)&oldid=1337873899](https://en.wikipedia.org/w/index.php?title=Rogue_(video_game)&oldid=1337873899).
- [8] Wikipedia Contributors. *Elite (video game)* — *Wikipedia, The Free Encyclopedia*. 2026. [Online; accessed 22-February-2026]. Dostupný z: [https://en.wikipedia.org/wiki/Elite_\(video_game\)](https://en.wikipedia.org/wiki/Elite_(video_game)).
- [9] *Godot Engine Documentation*. 2024. [Online; accessed 2026]. Dostupný z: <https://docs.godotengine.org>.
- [10] *GDScript Reference*. 2024. [Online; accessed 2026]. Dostupný z: <https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/>.
- [11] Pixel_Poem. *2D Pixel Dungeon Asset Pack*. 2023. [Online; accessed 2026]. Dostupný z: <https://pixel-poem.itch.io/dungeon-assetpack>.